

BAB I

PENGENALAN KONSEP BAHASA PEMROGRAMAN

1.1 Tujuan

- 1 Mengerti tentang konsep bahasa pemrograman
- 2 Membuat algoritma dari suatu permasalahan
- 3 Mengerti tentang flowchart
- 4 Membuat flowchart dari suatu permasalahan

1.2 Pengenalan Bahasa Pemrograman

Komputer adalah mesin yang dapat melaksanakan seperangkat perintah dasar (*instruction set*). Agar komputer dapat melakukan sesuatu hal, harus diberikan perintah yang dapat dilaksanakannya, yaitu dalam bentuk kumpulan perintah-perintah dasar. Program komputer adalah sekumpulan instruksi yang dikenal oleh komputer dan disusun menurut urutan yang logis untuk menyelesaikan suatu masalah. Bahasa pemrograman adalah bahasa yang dapat diterjemahkan menjadi kumpulan perintah-perintah dasar tersebut. Penerjemahan dilakukan oleh program komputer yang disebut kompilator. Sintaks dari bahasa pemrograman lebih mudah dipahami oleh manusia daripada sintaks perintah dasar. Namun tentu saja komputer hanya dapat melaksanakan perintah dasar itu. Maka di sinilah peran penting kompilator sebagai perantara antara bahasa pemrograman dengan perintah dasar. Kegiatan membuat program komputer dengan menggunakan bahasa pemrograman disebut pemrograman komputer. Untuk dapat membuat program komputer, harus dikuasai bahasa komputer. Berbagai bahasa komputer telah diciptakan untuk membantu manusia memprogram komputer. Berdasarkan tingkatannya dikenal beberapa tingkat bahasa pemrograman diantaranya:

- a. Bahasa Pemrograman dasar, seperti: Bahasa Mesin, Bahasa Assembly.
- b. Bahasa Pemrograman tingkat menengah, seperti: Bahasa C, Bahasa FORTH.
- c. Bahasa Pemrograman tingkat tinggi, seperti: Bahasa Pascal, C, FORTRAN, COBOL, dan lain-lain.

Bahasa pemrogram tingkat dasar, sebagai hirarki pertama, disebut juga bahasa generasi pertama, bahasa ini sangat tergantung pada jenis CPU yang dipakai oleh komputer. Bahasa ini sangat sulit dipelajari, karena sifatnya yang sangat tergantung pada mesin (*machine dependent*). Untuk menguasai bahasa ini, harus dipelajari dan dikuasai teknologi dan arsitektur komputer, matematika diskrit, elektronika, dan lain-lain. Bahasa ini masih menggunakan simbol-simbol yang bersifat mnemonic.

Bahasa pemrograman tingkat menengah, sifatnya sudah lebih mudah daripada bahasa pemrograman tingkat dasar, perintah-perintahnya sudah lebih mudah dibaca sebab sudah menggunakan huruf-huruf. Bahasa pemrograman tingkat tinggi adalah bahasa computer yang sudah mirip dengan bahasa manusia. Perintah-perintahnya sudah dibuat dalam bahasa yang mudah dimengerti manusia, seperti PRINT, WRITE, IF, THEN, ELSE, dan lain-lain.

1.3 Tahapan dalam Pemrograman

Langkah-langkah yang dilakukan dalam menyelesaikan masalah dalam pemrograman dengan komputer adalah:

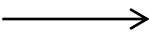
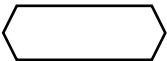
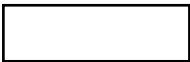
1. Defenisikan Masalah
 - a. Tentukan apa yang menjadi masalah
 - b. Tentukan data input yang diperlukan
 - c. Tentukan output yang diinginkan
2. Buat bagan dan struktur cara penyelesaian
 - a. Bagan secara global
 - b. Deskripsikan tugas masing-masing subprogram
3. Pilih metode penyelesaian
 - a. Pilih struktur data dan algoritma terbaik
4. Pengkodean
 - a. Pilih bahasa pemrograman yang sesuai
 - b. Menterjemahkan algoritma ke bahasa pemrograman

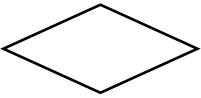
5. Mencari kesalahan
 - a. Kesalahan sintaks (penulisan program)
 - b. Kesalahan Pelaksanaan: Semantik, Logika, dan ketelitian
6. Uji dan verifikasi program
7. Dokumentasi Program
8. Pemeliharaan Program
 - a. Memperbaiki kekurangan yang ditemukan kemudian
 - b. Memodifikasi, karena perubahan spesifikasi

1.4 Pengertian Algoritma

Algoritma adalah urutan langkah-langkah logika yang menyatakan suatu tugas dalam menyelesaikan suatu masalah atau problem. Notasi yang digunakan untuk menuliskan algoritma ada dua, yaitu dengan menyatakan langkah-langkah algoritma dengan untaian kalimat deskriptif, atau dengan menggunakan diagram alir (flowchart). Diagram alir adalah simbol-simbol yang digunakan untuk menggambarkan sebuah pernyataan logika pemrograman serta aliran logika yang ditunjukkan dengan arah panah. Algoritma dengan flowchart dilakukan dengan simbol-simbol seperti yang diperlihatkan pada Tabel 1.1.

Tabel 1.1 Simbol-simbol flowchart

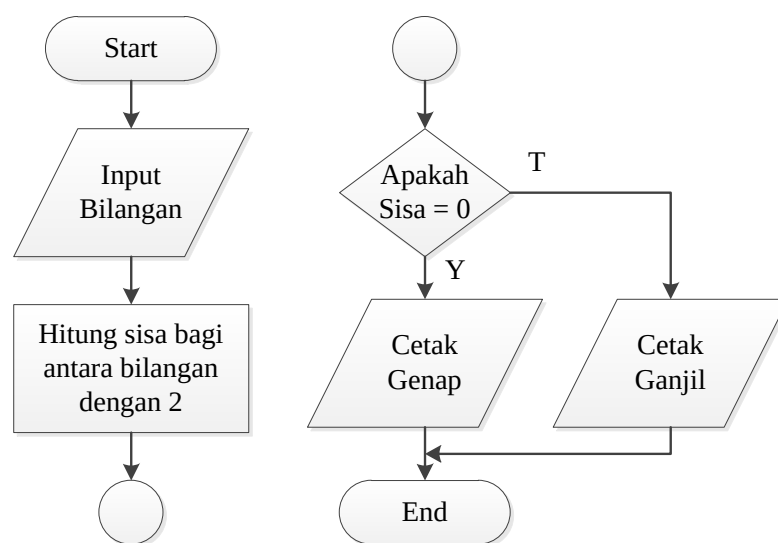
Simbol	Nama	Fungsi
	TERMINATOR	Permulaan/akhir program
	GARIS ALIR (FLOW LINE)	Arah aliran program
	PREPARATION	Proses inisialisasi/pemberian harga awal
	PROSES	Proses perhitungan/proses pengolahan data
	INPUT/OUTPUT DATA	Proses input/output data, parameter, informasi
	PREDEFINED PROCESS (SUB PROGRAM)	Permulaan sub program/proses menjalankan sub program

	DECISION	Perbandingan pernyataan, penyeleksian data yang memberikan pilihan untuk langkah selanjutnya
	ON PAGE CONNECTOR	Penghubung bagian-bagian flowchart yang berada pada satu halaman
	OFF PAGE CONNECTOR	Penghubung bagian-bagian flowchart yang berada pada halaman berbeda

Contoh:

Algoritma untuk menentukan jenis suatu bilangan, apakah bilangan ganjil atau bilangan genap. Flowchart program disajikan pada Gambar 1.1 dan Algoritma menggunakan kalimat deskriptif adalah sebagai berikut:

- Bagi bilangan dengan bilangan 2
- Hitung sisa hasil bagi pada langkah 1.
- Bila sisa hasil bagi sama dengan 0 maka bilangan itu adalah bilangan genap tetapi bila sisa hasil bagi sama dengan 1 maka bilangan itu adalah bilangan ganjil.



Gambar 1.1 Flowchart mencetak jenis bilangan

Latihan :

1. Buatlah sebuah rancangan program dengan menggunakan diagram alir, mencari luas persegi panjang.
2. Buat algoritma untuk menentukan apakah suatu bilangan merupakan bilangan prima atau bukan, kemudian buat flowchart untuk program tersebut !
3. Buat algoritma untuk mencetak N buah bilangan prima yang pertama, kemudian buat flowchart untuk program tersebut !

BAB II

SEKILAS TENTANG C

2.1 Tujuan

1. Menjelaskan sejarah dan ruang lingkup pemakaian bahasa C
2. Menjelaskan kelebihan dan kekurangan bahasa C
3. Menjelaskan proses kompilasi dan linking program C
4. Menjelaskan struktur penulisan bahasa C dan menjelaskan komponen-komponen program dalam contoh aplikasi sederhana

2.2 Sejarah dan Ruang Lingkup C

Akar dari bahasa C adalah bahasa BCPL yang dikembangkan oleh Martin Richards pada tahun 1967. Bahasa ini memberikan ide kepada Ken Thompson yang kemudian mengembangkan bahasa yang disebut dengan B pada tahun 1970. Perkembangan selanjutnya dari bahasa B adalah bahasa C oleh Dennis Ritchie sekitar tahun 1970-an di Bell Telephone Laboratories Inc. (sekarang adalah AT&T Bell Laboratories). Bahasa C pertama kali digunakan pada komputer Digital Equipment Corporation PDP-11 yang menggunakan sistem operasi UNIX.

C adalah bahasa yang standar, artinya suatu program yang ditulis dengan versi bahasa C tertentu akan dapat dikompilasi dengan versi bahasa C yang lain dengan sedikit modifikasi. Standar bahasa C yang asli adalah standar dari UNIX. Sistem operasi, kompiler C dan seluruh program aplikasi UNIX yang esensial ditulis dalam bahasa C. Patokan dari standar UNIX ini diambilkan dari buku yang ditulis oleh Brian Kernighan dan Dennis Ritchie berjudul "*The C Programming Language*", diterbitkan oleh Prentice-Hall tahun 1978. Deskripsi C dari Kernighan dan Ritchie ini kemudian dikenal secara umum sebagai "K&R C".

Kepopuleran bahasa C membuat versi-versi dari bahasa ini banyak dibuat untuk komputer mikro. Untuk membuat versi-versi tersebut menjadi standar, ANSI (*American National Standards Institute*) membentuk suatu komite (*ANSI committee X3J11*) pada tahun 1983 yang kemudian menetapkan standar ANSI untuk bahasa C. Standar ANSI ini didasarkan kepada standar UNIX yang

diperluas. Standar ANSI menetapkan sebanyak 32 buah kata-kata kunci (*keywords*) standar. Versi-versi bahasa C yang menyediakan paling tidak 32 kata-kata kunci ini dengan sintaks yang sesuai dengan yang ditentukan oleh standar, maka dapat dikatakan mengikuti standar ANSI. Buku ajar ini didasarkan pada bahasa C dari standar ANSI.

Pada saat ini C merupakan bahasa pemrograman yang sangat populer di dunia. Banyak pemrograman yang dibuat dengan bahasa C seperti assembler, interpreter, program paket, sistem operasi, editor, kompiler, program bantu, Word Star, Dbase, aplikasi untuk bisnis, matematika, dan game, bahkan ada pula yang menerapkannya untuk kecerdasan buatan.

Dalam beberapa literatur bahasa C digolongkan sebagai bahasa tingkat menengah. Penggolongan ke dalam bahasa tingkat menengah bukanlah berarti bahwa bahasa C lebih sulit dibandingkan dengan bahasa tingkat tinggi seperti PASCAL atau BASIC. Demikian juga bahasa C bukanlah bahasa yang berorientasi pada mesin seperti bahasa mesin dan assembly. Pada kenyataannya bahasa C mengkombinasikan elemen dalam bahasa tingkat tinggi dan bahasa tingkat rendah. Kemudahan dalam membuat program yang ditawarkan pada bahasa tingkat tinggi dan kecepatan eksekusi dari bahasa tingkat rendah merupakan tujuan diwujudkannya bahasa C.

2.3 Kelebihan dan Kelemahan C

Beberapa kelebihan dari bahasa C:

- Bahasa C tersedia hampir di semua jenis komputer, baik mikro, mini maupun komputer besar (mainframe computer).
- Kode bahasa C bersifat portabel. Suatu aplikasi yang ditulis dengan bahasa C untuk suatu komputer tertentu dapat digunakan di komputer lain hanya dengan sedikit modifikasi.
- Berbagai struktur data dan pengendalian proses disediakan dalam C sehingga memungkinkan untuk membuat program yang terstruktur. C memiliki struktur bahasa yang baik, selain mudah dipelajari juga memudahkan dalam pembuatan program, pelacakan kesalahan program dan menghasilkan dokumentasi program yang baik.

- Dibandingkan dengan bahasa mesin atau assembly, C jauh lebih mudah dipahami dan pemrogram tidak perlu mengetahui mesin komputer secara detil. Dengan demikian tidak akan menyita waktu yang terlampau banyak dalam menyelesaikan suatu masalah ke dalam bentuk program. Hal ini dikarenakan C merupakan bahasa yang berorientasi pada permasalahan, bukan berorientasi pada mesin.
- C memungkinkan memanipulasi data dalam bentuk bit maupun byte. Di samping itu juga memungkinkan untuk memanipulasi alamat dari suatu data atau pointer.

Adapun kelemahan bahasa C yang dirasakan oleh para pemula bahasa C:

- Banyaknya operator serta fleksibilitas penulisan program kadang-kadang membingungkan pemakai. Kalau tidak dikuasai sudah tentu akan menimbulkan masalah.
- Para pemrogram C tingkat pemula umumnya belum pernah mengenal pointer dan tidak terbiasa menggunakannya. Padahal kemampuan C justru terletak pada pointer.

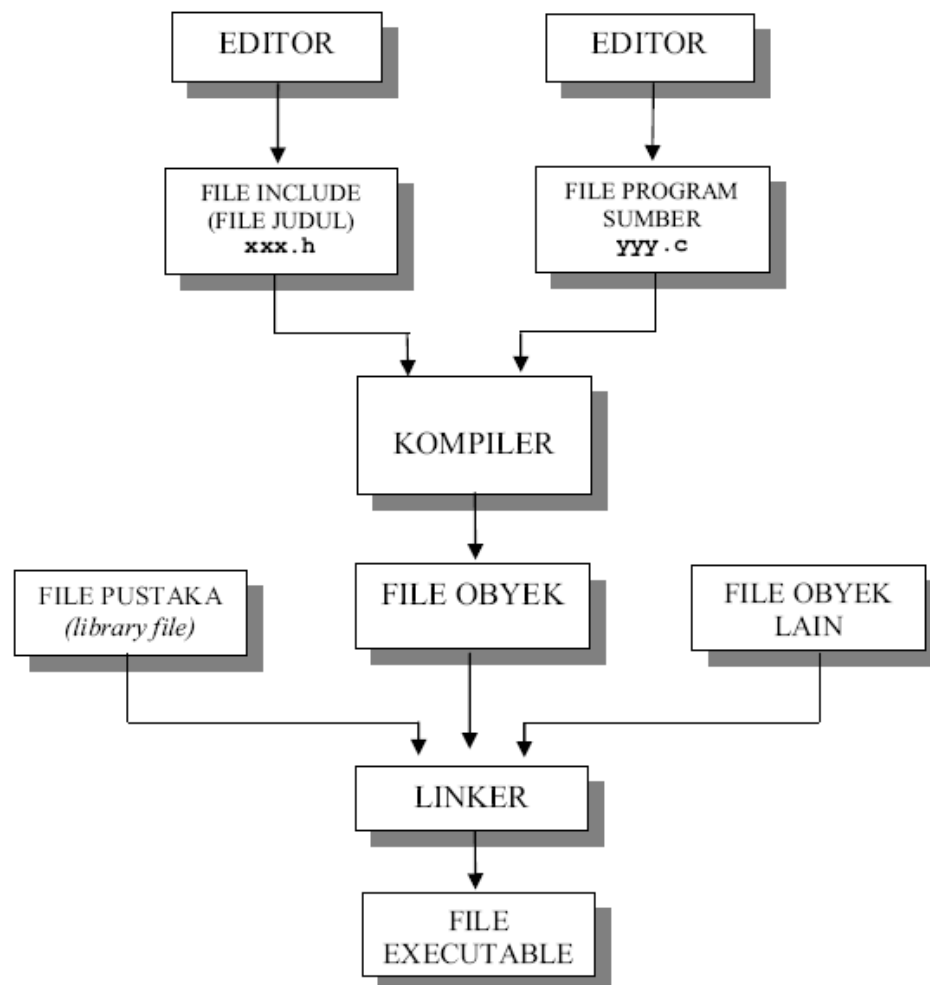
Kesulitan yang diuraikan di depan akan bersifat sementara saja. Kalau para pemula C mau mempelajarinya, sebenarnya tak ada yang dikatakan sulit sekali mengenai C. Mereka yang sudah terbiasa justru menyatakan bahwa bekerja dengan C sangat menyenangkan. Pepatah mengatakan “Di mana ada kemauan di situ ada jalan” dan “Jika tak kenal maka tak sayang”.

2.4 Proses Kompilasi dan Linking Program C

Agar suatu program dalam bahasa pemrograman dapat dimengerti oleh komputer, program haruslah diterjemahkan dahulu ke dalam kode mesin. Adapun penerjemah yang digunakan bisa berupa interpreter atau kompiler.

Interpreter adalah suatu jenis penerjemah yang menerjemahkan baris per baris instruksi untuk setiap saat. Keuntungan pemakaian interpreter, penyusunan program relatif lebih cepat dan bisa langsung diuji sekalipun masih ada beberapa kesalahan secara kaidah dalam program. Sedangkan kelemahannya, kecepatannya menjadi lambat sebab sebelum suatu instruksi dijalankan selalu harus

diterjemahkan terlebih dahulu. Selain itu, saat program dieksekusi, interpreter juga harus berada dalam memori. Jadi memori selalu digunakan baik untuk program maupun interpreter. Di samping itu, program sumber (*source program*) yaitu program aslinya tidak dapat dirahasiakan (orang lain selalu bisa melihatnya).



Gambar 2.1 Proses kompilasi-Linking dari program C

Keterangan Gambar:

- Pertama-tama program C ditulis dengan menggunakan editor. Program ini disimpan dalam file yang disebut file program sumber (dengan ciri utama memiliki ekstensi .c).
- File *include* (umumnya memiliki ekstensi .h, misalnya **stdio.h**, atau biasa disebut dengan file judul (*header file*)) berisi kode yang akan dilibatkan dalam program C (Pada program tertentu bisa saja tidak melibatkan file *include*).

- Berikutnya, kode dalam file program sumber maupun kode pada file include akan dikompilasi oleh kompiler menjadi kode obyek. Kode obyek ini disimpan pada file yang biasanya berekstensi **.obj**, atau **.o** (bergantung kepada lingkungan/environment sistem operasi yang dipakai). Kode obyek berbentuk kode mesin, oleh karena itu tidak dapat dibaca oleh pemrogram. Akan tetapi kode ini sendiri juga belum bisa dipahami komputer.
- Supaya bisa dimengerti oleh komputer, maka kode obyek bersama-sama dengan kode obyek yang lain (kalau ada) dan isi file pustaka (library file, yaitu file yang berisi rutin untuk melaksanakan tugas tertentu. File ini disediakan oleh pembuat kompiler, biasanya memiliki ekstensi **.lib**) perlu dikaitkan (*linking*) dengan menggunakan linker, membentuk sebuah program yang executable (program yang dapat dijalankan/dieksekusi secara langsung dalam lingkungan sistem operasi).
- Program hasil linker ini disimpan dalam file yang disebut file executable, yang biasanya berekstensi **.exe**.

Kebanyakan versi C yang beredar di pasaran menggunakan penerjemah berupa kompiler. Kompiler merupakan jenis penerjemah yang lain, dengan cara kerjanya yaitu menerjemahkan seluruh instruksi dalam program sekaligus. Proses pengkompilasian ini cukup dilakukan sekali saja. Selanjutnya hasil penerjemahan (setelah melalui tahapan yang lain) bisa dijalankan secara langsung, tanpa tergantung lagi oleh program sumber maupun kompilernya. Keuntungannya, proses eksekusi dapat berjalan dengan cepat, sebab tak ada lagi proses penerjemahan. Di samping itu, program sumber bisa dirahasiakan, sebab yang dieksekusi adalah program yang sudah dalam bentuk kode mesin. Sedangkan kelemahannya, proses pembuatan dan pengujian membutuhkan waktu relatif lebih lama, sebab ada waktu untuk mengkompilasi (menerjemahkan) dan ada pula waktu melakukan proses *linking*. Perlu pula diketahui, program akan berhasil dikompilasi hanya jika program tak mengandung kesalahan secara kaidah sama sekali.

Proses dari bentuk program sumber C (*source program*, yaitu program yang ditulis dalam bahasa C) hingga menjadi program yang *executable* (dapat dieksekusi secara langsung) ditunjukkan pada Gambar 2.1.

2.5 Struktur Penulisan Program C

Untuk dapat memahami bagaimana suatu program ditulis, maka struktur dari program harus dimengerti terlebih dahulu. Tiap bahasa komputer mempunyai struktur program yang berbeda. Struktur program memberikan gambaran secara luas, bagaimana bentuk program secara umum.

Program C pada hakekatnya tersusun atas sejumlah blok fungsi. Sebuah program minimal mengandung sebuah fungsi. Fungsi pertama yang harus ada dalam program C dan sudah ditentukan namanya adalah *main()*. Setiap fungsi terdiri atas satu atau beberapa pernyataan, yang secara keseluruhan dimaksudkan untuk melaksanakan tugas khusus. Bagian pernyataan fungsi (sering disebut tubuh fungsi) diawali dengan tanda kurung kurawal buka ({) dan diakhiri dengan tanda kurung kurawal tutup (}). Diantara kurung itu dapat dituliskan statemen-statemen program C. Namun pada kenyataannya, suatu fungsi bisa saja tidak mengandung pernyataan sama sekali. Walaupun fungsi tidak memiliki pernyataan, kurung kurawal haruslah tetap ada. Sebab kurung kurawal mengisyaratkan awal dan akhir definisi fungsi. Berikut ini adalah struktur dari program C.

<code>main()</code>	}	fungsi utama
{		
statemen-statemen;		
}		
 <code>fungsi_fungsi_lain()</code>	}	fungsi-fungsi lain yang ditulis oleh pemrogram
{		
statemen-statemen;		
}		

Bahasa C dikatakan sebagai bahasa pemrograman terstruktur karena strukturnya menggunakan fungsi-fungsi sebagai program-program bagiannya (*subroutine*). Fungsi-fungsi yang ada selain fungsi utama (*main()*) merupakan program-program bagian. Fungsi-fungsi ini dapat ditulis setelah fungsi utama atau diletakkan di file pustaka (*library*). Jika fungsi-fungsi diletakkan di file pustaka dan akan dipakai di suatu program, maka nama file judulnya (*header file*) harus dilibatkan dalam program yang menggunakannya dengan *preprocessor directive* berupa *#include*.

2.6 Pengenalan Program C

2.6.1. Pengenalan Fungsi-Fungsi Dasar

a. Fungsi *main()*

Pada program C, *main()* merupakan fungsi yang istimewa. Fungsi *main()* harus ada pada program, sebab fungsi inilah yang menjadi titik awal dan titik akhir eksekusi program. Tanda { di awal fungsi menyatakan awal tubuh fungsi dan sekaligus awal eksekusi program, sedangkan tanda } di akhir fungsi merupakan akhir tubuh fungsi dan sekaligus adalah akhir eksekusi program. Jika program terdiri atas lebih dari satu fungsi, fungsi *main()* biasa ditempatkan pada posisi yang paling atas dalam pendefinisian fungsi. Hal ini hanya merupakan kebiasaan. Tujuannya untuk memudahkan pencarian terhadap program utama bagi pemrogram. Jadi bukanlah merupakan suatu keharusan.

b. Fungsi *printf()*

Fungsi *printf()* merupakan fungsi yang umum dipakai untuk menampilkan suatu keluaran pada layar peraga. Untuk menampilkan tulisan

```
Selamat belajar bahasa C
```

misalnya, pernyataan yang diperlukan berupa:

```
printf("Selamat belajar bahasa C");
```

Pernyataan di atas berupa pemanggilan fungsi *printf()* dengan argumen atau parameter berupa string. Dalam C suatu konstanta string ditulis dengan diawali dan diakhiri tanda petik-ganda (“). Perlu juga diketahui pernyataan dalam C selalu diakhiri dengan tanda titik koma (;). Tanda titik koma dipakai sebagai tanda pemberhentian sebuah pernyataan dan bukanlah sebagai pemisah antara dua pernyataan.

Tanda \ pada string yang dilewatkan sebagai argumen *printf()* mempunyai makna yang khusus. Tanda ini bisa digunakan untuk menyatakan karakter khusus seperti karakter baris-baru ataupun karakter *backslash* (miring kiri). Jadi karakter seperti \n sebenarnya menyatakan sebuah karakter. Contoh karakter yang ditulis dengan diawali tanda \ adalah:

\” menyatakan karakter petik-ganda
\\ menyatakan karakter backslash
\t menyatakan karakter tab

Dalam bentuk yang lebih umum, format printf()

```
printf("string kontrol", daftar argumen);
```

dengan string kontrol dapat berupa satu atau sejumlah karakter yang akan ditampilkan ataupun berupa penentu format yang akan mengatur penampilan dari argumen yang terletak pada daftar argumen. Mengenai penentu format di antaranya berupa:

%d untuk menampilkan bilangan bulat (integer)
%f untuk menampilkan bilangan titik-mengambang (pecahan)
%c untuk menampilkan sebuah karakter
%s untuk menampilkan sebuah string

Contoh:

```
#include <stdio.h>
main( )
{
    printf("No : %d\n", 10);
    printf("Nama : %s\n", "Ali");
    printf("Nilai: %f\n", 80.5);
    printf("Huruf: %c\n", 'A');
}
```

2.6.2. Pengenalan Praprosesor #include

#include merupakan salah satu jenis pengarah praprosesor (*preprocessor directive*). Pengarah praprosesor ini dipakai untuk membaca file yang di antaranya berisi deklarasi fungsi dan definisi konstanta. Beberapa file judul disediakan dalam C. File-file ini mempunyai ciri yaitu namanya diakhiri dengan ekstensi **.h**. Misalnya pada program *#include <stdio.h>* menyatakan pada kompiler agar membaca file bernama stdio.h saat pelaksanaan kompilasi.

Bentuk umum #include:

```
#include "namafilename"
```

Bentuk pertama (`#include <namafile>`) mengisyaratkan bahwa pencarian file dilakukan pada direktori khusus, yaitu direktori file include. Sedangkan bentuk kedua (`#include "namafile"`) menyatakan bahwa pencarian file dilakukan pertama kali pada direktori aktif tempat program sumber dan seandainya tidak ditemukan pencarian akan dilanjutkan pada direktori lainnya yang sesuai dengan perintah pada sistem operasi.

Kebanyakan program melibatkan file *stdio.h* (file-judul I/O standard, yang disediakan dalam C). Program yang melibatkan file ini yaitu program yang menggunakan pustaka I/O (input-output) standar seperti *printf()*.

2.6.3 Komentar dalam Program

Untuk keperluan dokumentasi dengan maksud agar program mudah dipahami, biasanya pada program disertakan komentar atau keterangan mengenai program. Dalam C, suatu komentar ditulis dengan diawali dengan tanda `/*` dan diakhiri dengan tanda `*/`.

Contoh :

```
/*
Tanda ini adalah komentar
tidak masuk dalam eksekusi program */
#include <stdio.h>
main()
{
    printf("Coba\n");    //Ini adl program pertama
}
```

Kesimpulan :

- Akar dari bahasa C adalah bahasa BCPL yang dikembangkan oleh Martin Richards pada tahun 1967.
- Bahasa C pertama kali digunakan pada komputer Digital Equipment Corporation PDP-11 yang menggunakan sistem operasi UNIX.
- C adalah bahasa yang standar, artinya suatu program yang ditulis dengan versi bahasa C tertentu akan dapat dikompilasi dengan versi bahasa C yang lain dengan sedikit modifikasi. Standar bahasa C yang asli adalah standar dari UNIX.

- Interpreter adalah suatu jenis penerjemah yang menerjemahkan baris per baris instruksi untuk setiap saat, sedangkan kompiler merupakan jenis penerjemah cara kerjanya adalah menerjemahkan seluruh instruksi dalam program sekaligus.
- Program C pada hakekatnya tersusun atas sejumlah blok fungsi.
- Fungsi *main()* merupakan fungsi istimewa yang harus ada pada program, sebab fungsi inilah yang menjadi titik awal dan titik akhir eksekusi program.
- Fungsi *printf()* merupakan fungsi yang umum dipakai untuk menampilkan suatu keluaran pada layar peraga.
- *#include* merupakan salah satu jenis pengarah praprosesor (*preprocessor directive*) yang dipakai untuk membaca file yang di antaranya berisi deklarasi fungsi dan definisi konstanta.
- Untuk keperluan dokumentasi, di dalam program disertakan komentar yang ditulis dengan diawali dengan tanda */** dan diakhiri dengan tanda **/*.

Latihan :

Buatlah potongan program untuk soal-soal di bawah ini

1. Apakah keluaran dari program di bawah ini :


```
#include <stdio.h>
main()
{
    printf("The black dog was big. "); printf("The cow
jumped over the moon.\n");
}
```
2. Gunakan pernyataan *printf()* untuk menampilkan (di layar) nilai dari sebuah variabel (misalkan namanya = *sum*) yang bertipe integer.
3. Gunakan pernyataan *printf()* untuk menampilkan (di layar) string *"Welcome"* yang diikuti dengan sebuah perintah ganti baris.
4. Gunakan pernyataan *printf()* untuk menampilkan (di layar) sebuah karakter dari variabel yang bertipe karakter (misalkan namanya = **letter**).
5. Gunakan pernyataan *printf()* untuk menampilkan (di layar) nilai dari sebuah variabel float (misalkan namanya = **discount**).

6. Gunakan pernyataan *scanf()* untuk membaca masukan sebuah nilai desimal dari keyboard dan memasukkannya ke sebuah variabel integer (misalkan namanya = **sum**).
7. Gunakan pernyataan *scanf()* untuk membaca masukan nilai float dari keyboard dan memasukkannya ke sebuah variabel float (misalkan namanya = **discount_rate**).
8. Gunakan pernyataan *scanf()* untuk membaca masukan sebuah karakter dari keyboard dan memasukkannya ke sebuah variabel karakter (misalkan namanya = **opr**).

BAB III

DASAR-DASAR PEMROGRAMAN C

3.1 Tujuan

1. Menjelaskan tentang beberapa tipe data dasar (jenis dan jangkauannya)
2. Menjelaskan tentang variabel
3. Menjelaskan tentang konstanta
4. Menjelaskan tentang berbagai jenis operator dan pemakaiannya
5. Menjelaskan tentang instruksi I/O

3.2 Tipe Data Dasar

Data dapat dinyatakan sebagai suatu nilai dalam bentuk variabel atau konstanta. Variabel dapat memiliki nilai yang tidak tetap atau berubah-ubah, sedangkan konstanta memiliki nilai yang tetap selama eksekusi berlangsung.

Data berdasarkan jenisnya dibagi menjadi lima kelompok, yang dinamakan sebagai tipe data dasar. Kelima tipe data dasar yang dimaksud sebagai berikut.

- Bilangan bulat (integer)
- Bilangan real presisi-tunggal
- Bilangan real presisi-ganda
- Karakter
- Tak-bertipe *void()*

Tabel 3.1 memberikan informasi mengenai ukuran memori yang diperlukan dan kawasan dari masing-masing tipe data dasar.

Tabel 3.1. Ukuran memori untuk tipe data

Tipe	Total bit	Kawasan	Keterangan
char	8	-128 s/d 127	karakter
int	32	-2147483648 s/d 2147483647	bilangan integer
float	32	1.7E-38 s/d 3.4E+38	bilangan real presisi-tunggal
double	64	2.2E-308 s/d 1.7E+308	bilangan real presisi-ganda

Kata-kunci yang berkaitan dengan tipe data dasar secara berurutan di antaranya adalah *int* (*short int*, *long int*, *signed int* dan *unsigned int*), *float*, *double*, dan *char*. Untuk tipe data *short int*, *long int*, *signed int* dan *unsigned int*, maka ukuran memori yang diperlukan serta kawasan dari masing-masing tipe data, dapat dilihat pada Tabel 3.2. .

Tabel 3.2 Ukuran memori untuk tipe data *int*

Tipe	Total bit	Kawasan	Keterangan
short int	16	-32768 s/d 32767	short integer
long int	32	-2147483648 s/d 2147483647	long integer
signed int	32	-2147483648 s/d 2147483647	biasa disingkat dengan int
unsigned int	32	0 s/d 4294967295	bilangan int tak bertanda

Catatan :

Ukuran dan kawasan dari masing-masing tipe data adalah bergantung pada jenis mesin yang digunakan (misalnya mesin 16 bit bisa jadi memberikan hasil berbeda dengan mesin 32 bit).

3.3 Variabel

3.3.1 Aturan Pendefinisian Variabel

Aturan penulisan pengenalan untuk sebuah variabel, konstanta atau fungsi yang didefinisikan oleh pemrogram adalah sebagai berikut:

- Pengenal harus diawali dengan huruf (*A..Z*, *a..z*) atau karakter garis bawah (*_*).
- Selanjutnya dapat berupa huruf, digit (*0..9*) atau karakter garis bawah atau tanda dollar (*\$*).
- Panjang pengenalan boleh lebih dari 31 karakter, tetapi hanya 31 karakter pertama yang akan dianggap berarti.
- Pengenal tidak boleh menggunakan nama yang tergolong sebagai kata-kata cadangan (*reserved words*) seperti *int*, *if*, *while* dan sebagainya.

3.3.2 Mendeklarasikan Variabel

Variabel digunakan dalam program untuk menyimpan suatu nilai, dan nilai yang ada padanya dapat diubah-ubah selama eksekusi program berlangsung. Variabel yang akan digunakan dalam program haruslah dideklarasikan terlebih dahulu. Pengertian deklarasi di sini berarti memesan memori dan menentukan jenis data yang bisa disimpan di dalamnya.

Bentuk umum deklarasi variabel:

```
tipe daftar-variabel;
```

Pada pendeklarasian variabel, daftar-variabel dapat berupa sebuah variabel atau beberapa variabel yang dipisahkan dengan koma. Contoh:

```
int var_bulat1;  
float var_pecahan1, var_pecahan2;
```

3.3.3 Memberikan Nilai ke Variabel

Untuk memberikan nilai ke variabel yang telah dideklarasikan, maka bentuk umum pernyataan yang digunakan adalah :

```
nama_variabel = nilai;
```

Contoh:

```
int var_bulat = 10;  
double var_pecahan = 10.5;
```

3.3.4 Inisialisasi Variabel

Adakalanya dalam penulisan program, setelah dideklarasikan, variabel langsung diberi nilai awal. Sebagai contoh yaitu variabel **nilai** :

```
int nilai;  
nilai = 10;
```

Dua pernyataan di atas sebenarnya dapat disingkat melalui pendeklarasian yang disertai penugasan nilai, sebagai berikut :

```
int nilai= 10;
```

Cara seperti ini banyak dipakai dalam program C, di samping menghemat penulisan pernyataan, juga lebih memberikan kejelasan, khususnya untuk variabel yang perlu diberi nilai awal (diinisialisasi).

3.4 Konstanta

Konstanta menyatakan nilai yang tetap. Berbeda dengan variabel, suatu konstanta tidak dideklarasikan. Namun seperti halnya variabel, konstanta juga memiliki tipe. Penulisan konstanta mempunyai aturan tersendiri, sesuai dengan tipe masing-masing.

- Konstanta karakter misalnya ditulis dengan diawali dan diakhiri dengan tanda petik tunggal, contohnya : 'A' dan '@'.
- Konstanta integer ditulis dengan tanda mengandung pemisah ribuan dan tak mengandung bagian pecahan, contohnya : -1 dan 32767.
- Konstanta real (float dan double) bisa mengandung pecahan (dengan tanda berupa titik) dan nilainya bisa ditulis dalam bentuk eksponensial (menggunakan tanda *e*), contohnya : 27.5f (untuk tipe float) atau 27.5 (untuk tipe *double*) dan 2.1e+5 (maksudnya $2,1 \times 10^5$).
- Konstanta string merupakan deretan karakter yang diawali dan diakhiri dengan tanda petik-ganda (""), contohnya : "Pemrograman Dasar C".

3.5 Operator

Operator merupakan simbol atau karakter yang biasa dilibatkan dalam program untuk melakukan sesuatu operasi atau manipulasi, seperti menjumlahkan dua buah nilai, memberikan nilai ke suatu variabel, membandingkan kesamaan dua buah nilai. Sebagian operator C tergolong sebagai operator binary, yaitu operator yang dikenakan terhadap dua buah nilai (*operand*). Contoh :

$$a + b$$

Simbol + merupakan operator untuk melakukan operasi penjumlahan dari kedua *operand*-nya (yaitu a dan b). Karena operator penjumlahan melibatkan dua operand ini tergolong sebagai operator binary.

$$-c$$

Simbol - (minus) juga merupakan operator. Simbol ini termasuk sebagai operator unary, yaitu operator yang hanya memiliki sebuah operand (yaitu c pada contoh ini).

3.5.1 Operator Aritmatika

Operator untuk operasi aritmatika yang tergolong sebagai operator binary adalah :

- * perkalian
- / pembagian
- % sisa pembagian
- + penjumlahan
- pengurangan

Adapun operator yang tergolong sebagai operator unary.

- tanda minus
- + tanda plus

Contoh pemakaian operator aritmatika misalnya untuk memperoleh nilai diskriminan dari suatu persamaan kuadrat : $D = b^2 - 4ac$

```
/* File program : diskrim.c
Menghitung diskriminan pers kuadrat  $ax^2 + bx + c = 0$ 
*/
# include <stdio.h>
main()
{
    float a,b,c,d;
    a = 3.0f;
    b = 4.0f;
    c = 7.0f;
    d = b*b-4*a*c;
    printf("Diskriminan = %f\n", d);
}
```

Contoh eksekusi :

Diskriminan = -68.000000

Operator yang telah dituliskan di atas, yang perlu diberi penjelasan lebih lanjut adalah operator sisa pembagian. Beberapa contoh berikut kiranya akan memperjelas makna dari operator ini .

- Sisa pembagian bilangan 7 dengan 2 adalah 1 ($7 \% 2 \rightarrow 1$)
- Sisa pembagian bilangan 6 dengan 2 adalah 0 ($6 \% 2 \rightarrow 0$)
- Sisa pembagian bilangan 8 dengan 3 adalah 2 ($8 \% 3 \rightarrow 2$)

Kegunaan operator ini diantaranya bisa dipakai untuk menentukan suatu bilangan bulat termasuk ganjil atau genap, berdasarkan logika : “Jika bilangan habis dibagi dua (sisanya nol), bilangan termasuk genap. Sebaliknya, termasuk ganjil”.

3.5.2 Operator Penurunan dan Peningkatan

Masih berkaitan dengan operasi aritmatika, C menyediakan operator yang disebut sebagai operator peningkatan dan operator penurunan, yaitu :

`++` operator peningkatan

`--` operator penurunan

Operator peningkatan digunakan untuk menaikkan nilai variabel sebesar satu. Penempatan operator terhadap variabel dapat dilakukan di muka atau di belakangnya, contohnya :

```
x = x+1;
```

```
y = y-1;
```

Bisa ditulis menjadi :

```
++x;
```

```
--y;
```

atau :

```
x++;
```

```
y--;
```

bergantung pada kondisi yang dibutuhkan oleh pemrogram. Di bawah ini adalah contoh yang akan menunjukkan perbedaan pemakaian dan hasil dari `++x` dengan `x++` (atau pemakaian `y--` dengan `--y`).

```
/* File program : pre_post.c
Contoh penggunaan pre & post Increment operator */
#include <stdio.h>
main()
{
    int count = 0, loop;
    loop = ++count; /* count=count+1; loop=count; */
    printf("loop = %d, count = %d\n", loop, count);
    loop = count++; /* loop=count; count=count+1; */
    printf("loop = %d, count = %d\n", loop, count);
}
```

Contoh eksekusi :

loop = 1, count = 1

loop = 1, count = 2

3.5.3 Prioritas Operator Aritmatika

Tabel di bawah ini memberikan penjelasan mengenai prioritas dari masing-masing operator. Operator yang mempunyai prioritas tinggi akan diutamakan dalam hal pengerjaan dibandingkan dengan operator yang memiliki prioritas lebih rendah.

Tabel 3.3 Tabel prioritas operator aritmatika dan urutan pengerjaannya

PRIORITAS	OPERATOR	URUTAN PENGGERJAAN
Tertinggi	()	dari kiri ke kanan
	! ++ -- + -	dari kanan ke kiri *)
Terendah	* / %	dari kiri ke kanan
	+ -	dari kiri ke kanan *)
	= += -= *= /=	dari kanan ke kiri
	%=	

*) Bentuk **unary +** dan **unary -** memiliki prioritas yang lebih tinggi daripada bentuk **binary +** dan **binary -**

3.5.4 Operator Penugasan

Operator penugasan (*assignment operator*) digunakan untuk memindahkan nilai dari suatu ungkapan (*expression*) ke suatu pengenalan. Operator pengerjaan yang umum digunakan dalam bahasa pemrograman, termasuk bahasa C adalah operator sama dengan (=). Contohnya:

```
fahrenheit = celcius * 1.8 + 32;
```

Maka '=' adalah operator penugasan yang akan memberikan nilai dari ungkapan : *celcius * 1.8 + 32* kepada variabel *fahrenheit*.

Bahasa C juga memungkinkan dibentuknya statemen penugasan menggunakan operator pengerjaan jamak dengan bentuk sebagai berikut.

pengenal1 = pengenal2 = ... = ungkapan ;

Misalnya:

`a = b = 15;`

maka nilai variabel 'a' akan sama dengan nilai variabel 'b' akan sama dengan 15.

3.5.5 Operator Kombinasi (Pemendekan)

C menyediakan operator yang dimaksudkan untuk memendekkan penulisan operasi penugasan. Tabel 3.4 memberikan seluruh kemungkinan operator kombinasi dalam suatu pernyataan serta pernyataan padanannya.

Tabel 3.4 Seluruh kemungkinan operator kombinasi dan padanannya

<code>x += 2;</code>	kependekan dari	<code>x = x + 2;</code>
<code>x -= 2;</code>	kependekan dari	<code>x = x - 2;</code>
<code>x *= 2;</code>	kependekan dari	<code>x = x * 2;</code>
<code>x /= 2;</code>	kependekan dari	<code>x = x / 2;</code>
<code>x %= 2;</code>	kependekan dari	<code>x = x % 2;</code>
<code>x <=< 2;</code>	kependekan dari	<code>x = x << 2;</code>
<code>x >>= 2;</code>	kependekan dari	<code>x = x >> 2;</code>
<code>x &= 2;</code>	kependekan dari	<code>x = x & 2;</code>
<code>x = 2;</code>	kependekan dari	<code>x = x 2;</code>
<code>x ^= 2;</code>	kependekan dari	<code>x = x ^ 2;</code>

3.6 Menampilkan Data ke Layar

Untuk keperluan menampilkan data/informasi, C menyediakan sejumlah fungsi. Beberapa di antaranya adalah berupa *printf()* dan *putchar()*.

3.6.1. Fungsi *printf()*

Fungsi *printf()* merupakan fungsi yang paling umum digunakan dalam menampilkan data. Berbagai jenis data dapat ditampilkan ke layar dengan memakai fungsi ini. Bentuk umum pernyataan *printf()* :

`printf("string kontrol", argumen1, argumen2, ...);`

String kontrol dapat berupa keterangan yang akan ditampilkan pada layar beserta penentu format (seperti %d, %f, %c). Penentu format dipakai untuk memberi tahu kompiler mengenai jenis data yang akan ditampilkan. Argumen sesudah string kontrol (argumen1, argumen2,...) adalah data yang akan ditampilkan ke layar. Argumen ini dapat berupa variabel, konstanta dan bahkan ungkapan. Misal :

```
printf("%d",20); /* argumen berupa konstanta */
printf("%d",a); /*argumen berupa variabel */
printf("%d",a+20);/*argumen berupa ungkapan */
```

Penentu format untuk data string atau karakter :

%c	untuk menampilkan sebuah karakter
%s	untuk menampilkan sebuah string

Untuk menampilkan data bilangan, penentu format yang dipakai berupa salah satu dari bentuk dalam Tabel 3.5.

Tabel 3.5 Penentu format pada *printf()*

%u	untuk menampilkan data bilangan tak bertanda (<i>unsigned</i>) dalam bentuk decimal
%d	untuk menampilkan bilangan integer bertanda (<i>signed</i>) dalam bentuk decimal
%i	
%o	untuk menampilkan bilangan bulat tak bertanda dalam bentuk oktal.
%x	untuk menampilkan bilangan bulat tak bertanda dalam bentuk heksadesimal (%x → notasi yang dipakai : a, b, c, d, e dan f sedangkan %X → notasi yang dipakai : A, B, C, D, E dan F)
%X	
%f	untuk menampilkan bilangan real dalam notasi : dddd.dddddd
%e	untuk menampilkan bilangan real dalam notasi eksponensial
%E	
%g	untuk menampilkan bilangan real dalam bentuk notasi seperti %f,%E atau %F bergantung pada kepresisian data (digit 0 yang tak berarti tak akan ditampilkan)
%G	
l	Merupakan awalan yang digunakan untuk %d,%u,%x,%X,%o untuk menyatakan long int (misal %ld). Jika diterapkan bersama %e,%E,%f,%F,%g atau %G akan menyatakan <i>double</i>
L	Merupakan awalan yang digunakan untuk %f,%e,%E,%g dan %G untuk menyatakan long <i>double</i>
h	Merupakan awalan yang digunakan untuk %d,%i,%o,%u,%x, atau %X, untuk menyatakan <i>short int</i>

Contoh di bawah ini akan menjelaskan perbedaan format `%g`, `%e` dan `%f` dalam menampilkan bilangan *real*.

```
/*File program : form_efg.c
Perbedaan format %g, %e dan %f */
#include <stdio.h>
main()
{
    float x = 251000.0f;
    printf("Format e => %e\n", x);
    printf("Format f => %f\n", x);
    printf("Format g => %g\n", x);
}
```

Contoh eksekusi :

Format e => 2.510000e+005

Format f => 251000.000000

Format g => 251000

Tampak bahwa penentu format `%e` menampilkan bilangan dalam bentuk eksponensial. Jika penentu format yang digunakan berupa `%f`, bagian pecahan secara *default* akan ditampilkan dalam bentuk 6 digit. Sedangkan jika digunakan penentu format `%g`, maka digit yang tak berarti tak akan ditampilkan.

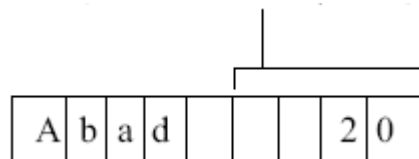
Untuk menentukan panjang medan yang disediakan bagi tampilan data, maka sesudah tanda `%` dalam penentu format dapat disisipi dengan bilangan bulat yang menyatakan panjang medan.

- Untuk data yang berupa bilangan bulat, misal pada :

```
printf("Abad %4d", 20);
```

`%4d` menyatakan medan untuk menampilkan bilangan 20 adalah sepanjang 4 karakter.

```
printf("Abad %4d", 20);
```



- Untuk data yang berupa bilangan real, spesifikasi medannya berupa

m.n

m = panjang medan

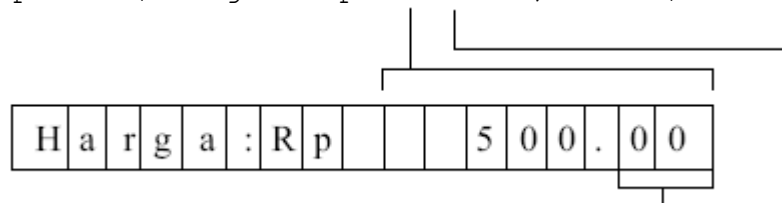
n = jumlah digit pecahan

Contoh pada pernyataan :

```
printf("Harga : Rp %8.2f\n", 500.0);
```

%8.2f menyatakan panjang medan dari bilangan real yang akan ditampilkan adalah 8 karakter dengan jumlah digit pecahan 2 buah.

```
printf("Harga : Rp %8.2f\n", 500.0);
```



Kalau hanya jumlah digit pecahan yang perlu ditentukan, panjang medan tak perlu disertakan, misal :

```
printf("%.2f\n", 600.0);
```

```
printf("%.2f\n", 7500.25);
```

hasilnya :

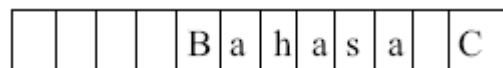
600.00

7500.25

- Untuk data yang berupa string, contoh :

```
printf("%12s", "Bahasa C");
```

maka akan ditampilkan sebagai berikut



Tampak dalam berbagai jenis data di atas, penentu format yang mengandung panjang medan, secara default akan menampilkan data dalam bentuk rata kanan terhadap panjang medan yang diberikan. Untuk data string yang biasanya dikehendaki untuk ditampilkan dalam bentuk rata kiri, maka sesudah tanda % pada penentu format %s perlu disisipkan tanda - (minus),

contoh :

```
printf("%-12s", "Bahasa C");
```

menyatakan bahwa string akan ditampilkan dalam medan dengan panjang 12 karakter dan diatur rata kiri. Sehingga tampilan di atas berubah menjadi :

B	a	h	a	s	a		C				
---	---	---	---	---	---	--	---	--	--	--	--

```
/* File program : formatpjpg.c
Contoh penggunaan format panjang medan data */
#include <stdio.h>
main()
{
    int nilai1 = 20;
    float nilai2 = 500.0f;
    printf("Abad %5d\n", nilai1);
    printf("%10.2f\n", nilai2);
    printf("%10s\n", "Bahasa C");
    printf("%-10s\n", "Bahasa C");
}
```

Contoh eksekusi :

```
Abad          20
              500.00
              Bahasa C
Bahasa C
```

3.6.2. Fungsi *putchar()*

Fungsi *putchar()* digunakan khusus untuk menampilkan sebuah karakter di layar. Penampilan karakter tidak diakhiri dengan perpindahan baris.

Contoh :

```
putchar('A');
```

menghasilkan keluaran yang sama dengan

```
printf("%c", 'A');
```

3.7 Memasukan Data dari Keyboard

Data dapat dimasukan lewat keyboard saat eksekusi berlangsung. Untuk keperluan ini, C menyediakan sejumlah fungsi, di antaranya adalah *scanf()*, *getchar()*.

3.7.1. Fungsi *scanf()*

Fungsi *scanf()* merupakan fungsi yang dapat digunakan untuk memasukkan berbagai jenis data. Misalnya untuk memasukkan data jari-jari lingkaran pada contoh program **lingkaran.c**, maka penulisan

```
radius = 20;
```

dapat diganti menjadi

```
scanf("%f",&radius);
```

Selengkapnya, terlihat dalam contoh program di bawah ini.

```
/* File program : lingkaran.c
Menghitung keliling dan luas lingkaran */
#include <stdio.h>
main()
{
    double radius, keliling, luas;
    printf("Masukkan jari-jari lingkaran : ");
    scanf("%lf",&radius);
    keliling = 2 * 3.14 * radius; /* PI = 3.14 */
    luas = 0.5 * 3.14 * radius * radius;
    printf("\nData lingkaran\n");
    printf("Jari-jari = %8.2lf\n", radius);
    printf("Keliling = %8.2lf\n", keliling);
    printf("Luas = %8.2lf\n", luas);
}
```

Contoh eksekusi :

Masukkan jari-jari lingkaran = 5

Data lingkaran

Jari-jari = 5.00

Keliling = 31.40

Luas = 39.25

Bentuk *scanf()* sesungguhnya menyerupai fungsi *printf()*. Fungsi ini melibatkan penentu format yang pada dasarnya sama digunakan pada *printf()*. Secara umum bentuk *scanf()* adalah sebagai berikut :

```
scanf("string kontrol", daftar_argumen);
```

Dengan string kontrol dapat berupa :

- Penentu format
- Karakter spasi-putih (*white-space*)
- Karakter bukan spasi-putih

Penentu format menyatakan jenis data yang akan dibaca. Pada *scanf()* penentu format dapat berupa salah satu di antara yang ada pada Tabel 3.6.

Tabel 3.6 Penentu format *scanf()*

%c	membaca sebuah karakter
%s	membaca sebuah string (dibahas pada bab vii)
%i atau %d	membaca sebuah integer desimal
%e atau %f	membaca sebuah bilangan real (bisa dalam bentuk eksponensial)
%o	membaca sebuah integer oktal
%x	membaca sebuah integer heksadesimal
%u	membaca sebuah integer tak bertanda
l	awalan untuk membaca data long int (misal : %ld) atau untuk membaca data double (misal : %lf)
L	awalan untuk membaca data long double (misal : %Lf)
h	awalan untuk membaca data short int

Pada bentuk *scanf()*, **daftar_argumen** dapat berupa satu atau beberapa argumen dan haruslah berupa **alamat**. Misalnya hendak membaca bilangan real dan ditempatkan ke variabel *radius*, maka yang ditulis dalam *scanf()* adalah alamat dari **radius**. Untuk menyatakan alamat dari variabel, di depan variabel dapat ditambahkan tanda & (tanda & dinamakan sebagai operator alamat). Sehingga *&radius* menyatakan alamat dari **radius**. Dalam bentuk yang lengkap :

```
scanf("%f", &radius);
```

berarti (bagi komputer) : “bacalah sebuah bilangan real (%f) dan tempatkan ke alamat dari **radius** (*&radius*)”.

3.7.2. Fungsi *getchar()*

Fungsi *getchar()* digunakan khusus untuk menerima masukan berupa sebuah karakter dari keyboard. Contoh :

```
c = getchar();  
scanf("%c", &c);
```

maka variabel **c** akan berisi karakter yang diketikkan oleh user atau *EOF (end of file)* jika ditemui akhir dari file.

Kesimpulan :

- Data merupakan suatu nilai yang bisa dinyatakan dalam bentuk konstanta atau variabel
- Konstanta menyatakan nilai yang tetap, sedangkan variabel menyatakan nilai yang dapat diubah-ubah selama eksekusi berlangsung
- Variabel yang akan digunakan haruslah dideklarasikan terlebih dahulu, adakalanya langsung dideklarasikan sekaligus diberi nilai (diinisialisasi)
- Operator merupakan simbol atau karakter yang biasa dilibatkan dalam program untuk melakukan sesuatu operasi atau manipulasi
- Operator yang terkait dengan operasi aritmatika antara lain adalah operator aritmatika, operator penurunan dan kenaikan, operator penugasan (*assignment*) dan operator kombinasi (pemendekan)
- Untuk menampilkan data/informasi ke layar digunakan fungsi *printf()* dan *putchar()*
- Untuk memasukkan data melalui keyboard saat eksekusi berlangsung digunakan fungsi *scanf()* dan *getchar()*

Latihan :

1. Mengapa nama-nama variabel di bawah ini tidak valid ?

- (a) *value\$sum*
- (b) *exit flag*
- (c) *3lotsofmoney*
- (d) *char*

2. Berapakah hasil akhir dari program berikut :

```
#include  
<stdio.h>
```

```

main()
{
    int a = 22;
    a =
    a +
    5; a
    = a-
    2;
    printf("a = %d\n", a);
}

```

3. Berapakah nilai x setelah pernyataan-pernyataan berikut dijalankan, apabila x bertipe *int* :

- (a) $x = (2 + 3) - 10 * 2;$
- (b) $x = (2 + 3) - (10 * 2);$
- (c) $x = 10 \% 3 * 2 + 1;$

4. Nyatakan dalam bentuk pernyataan :

(a) $y = bx^2 + 0,5x - c$

(b) $y = \frac{0,3xz}{2a}$

5. Apa hasil eksekusi dari program berikut :

```

#include
<stdio.h>
main()
{
    char kar = 'A';
    kar = kar + 32;
    printf("%c\n", k
ar);
}

```


BAB IV

PENGAMBILAN KEPUTUSAN

4.1 Tujuan

1. Menjelaskan tentang operator kondisi (operator relasi dan logika)
2. Menjelaskan penggunaan pernyataan *if*
3. Menjelaskan penggunaan pernyataan *if-else*
4. Menjelaskan penggunaan pernyataan *if* dalam *if*
5. Menjelaskan penggunaan pernyataan *else-if*
6. Menjelaskan penggunaan pernyataan *switch*

4.2 Operator Kondisi

Banyak persoalan yang diperlukan untuk membuat keputusan. Contoh yang sederhana berupa cara mengatur agar komputer bisa menyimpulkan bahwa suatu bilangan merupakan bilangan genap atau bilangan ganjil. Untuk keperluan pengambilan keputusan semacam itu, C menyediakan beberapa jenis pernyataan, berupa

- Pernyataan *if*
- Pernyataan *if-else*, dan
- Pernyataan *switch*

Pernyataan-pernyataan tersebut memerlukan suatu kondisi, sebagai basis dalam pengambilan keputusan. Kondisi umum yang dipakai berupa keadaan benar dan salah. Oleh karena itu pembahasan pada bab ini akan diawali dengan pengenalan operator yang membentuk kondisi benar dan salah.

Operator yang digunakan untuk menghasilkan kondisi benar dan salah, bisa berupa operator relasi dan bisa juga berupa operator logika. Berikut ini dibahas masing-masing jenis operator serta tabel prioritas masing-masing operator.

4.2.1. Operator Relasi

Operator relasi biasa dipakai untuk membandingkan dua buah nilai. Hasil perbandingan berupa keadaan benar atau salah. Keseluruhan operator relasi pada C ditunjukkan pada Tabel 4.1.

Tabel 4.1. Operasi relasi

Operator	Makna
>	Lebih dari
>=	Lebih dari atau sama dengan
<	Kurang dari
<=	Kurang dari atau sama dengan
==	Sama dengan
!=	Tidak sama dengan

Khususnya untuk operator relasi sama dengan (==) harap dibedakan dengan operator (=) yang merupakan operator penugasan (assignment). Contoh:

Pembandingan	Hasil
1 > 2	Salah
1 < 2	Benar
A == 1	Benar, jika A bernilai 1 Salah, jika A tidak bernilai 1
'A' < 'B'	Benar, karena kode ASCII untuk karakter 'A' kurang dari kode ASCII untuk karakter 'B' *)
kar == 'Y'	Benar, jika kar berisi 'Y' Salah, jika kar tidak berisi 'Y'

*) Dalam daftar ASCII standar, kode untuk karakter 'A' = 65 sedangkan karakter 'B' = 66, 'C' = 67, 'D' = 68 dan seterusnya sampai dengan karakter 'Z' = 90.

4.2.2. Operator Logika

Operator logika biasa dipakai untuk menghubungkan ekspresi relasi. Keseluruhan operator logika ditunjukkan pada tabel 4.2.

Tabel 4.2. Operator logika

Operator	Makna
&&	dan (AND)
	atau (OR)
!	tidak (NOT)

Bentuk pemakaian operator && dan || adalah

operand1 operator operand2

Baik **operand1** maupun **operand2** dapat berupa ekspresi relasi ataupun ekspresi logika. Hasil ekspresi bisa bernilai benar atau salah. Pada C nilai hasil dari sebuah ekspresi relasi atau ekspresi logika jika dinyatakan dengan angka adalah :

Salah → nilai = 0

Benar → nilai != 0 (misalnya nilai = 1)

Tabel 4.3 memberikan penjelasan hasil operasi ekspresi logika yang menggunakan operator && maupun || untuk berbagai kemungkinan keadaan operand-nya.

Tabel 4.3. Kemungkinan pada operasi logika && dan ||

Operand1	Operand2	Hasil	
			&&
Salah	Salah	0	0
Salah	Benar	1	0
Benar	Salah	1	0
Benar	Benar	1	1

Tampak bahwa operator **atau** (`||`) menghasilkan nilai `1` jika ada operand yang benar. Hasil berupa `0` jika semua operand adalah salah. Adapun operator logika **dan** (`&&`) memberikan hasil `1` hanya jika kedua operand adalah benar.

Beberapa contoh ekspresi logika di antaranya :

- `(kar > 'A') && (kar < 'Z')`

Hasil operasi logika `&&` adalah benar hanya jika `kar > 'A'` dan `kar < 'Z'` (dalam hal ini yang diperbandingkan adalah kode ASCII dari karakter tsb).

- `(pilihan == 'Y') || (pilihan == 'y')`

Hasil operasi logika `||` adalah benar jika pilihan berupa `'Y'` atau `'y'`

Sedangkan bentuk pemakaian operator logika `!` adalah :

<code>!operand</code>

dengan **operand** dapat berupa ekspresi logika ataupun ekspresi relasi.

Hasil operasi `!` bernilai :

- `1` jika **operand** bernilai salah
- `0` jika **operand** bernilai benar

Perhatikan contoh potongan program di bawah ini :

```
if (!sudah_benar)
    printf("Masukan Anda salah!\n");
```

Pada contoh potongan program di atas, dilakukan pengecekan kondisi terhadap nilai dari variabel **sudah_benar**. Jika variabel **sudah_benar** bernilai `0`, maka kondisi **!sudah_benar** akan bernilai benar (*true*) sehingga instruksi :

```
printf("Masukan Anda salah!\n");
```

akan diproses. Penjelasan lebih rinci tentang pengecekan kondisi dengan pernyataan `if` dibahas pada sub bab 3.2.

4.2.3. Prioritas Operator Logika dan Relasi

Tabel 4.4 memberikan penjelasan singkat mengenai prioritas di antara berbagai operator logika dan operator relasi.

Tabel 4.4 Prioritas operator logika dan relasi

Tertinggi :	!			
	>	>=	<	<=
	= =	!=		
	&&			
Terendah :				

Berdasarkan prioritas yang ditunjukkan pada tabel 3-4, maka ekspresi seperti

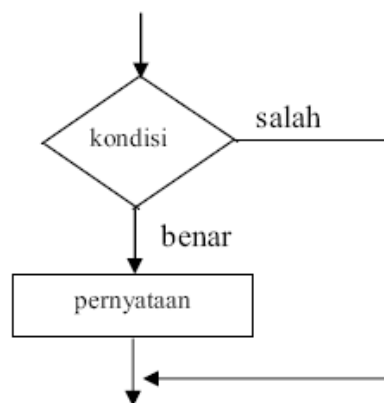
```
(kar > 'A') && (kar < 'Z')
```

sama saja kalau ditulis menjadi

```
kar > 'A' && kar < 'Z'
```

Hanya saja penulisan dengan menggunakan tanda kurung akan lebih memberikan kejelasan.

4.3 Pernyataan *if*



Gambar 4.1. Diagram alir *if*

Pernyataan *if* mempunyai bentuk umum :

```
if (kondisi)  
pernyataan;
```

Bentuk ini menyatakan :

- Jika kondisi yang diseleksi adalah benar (bernilai logika = 1), maka pernyataan yang mengikutinya akan diproses.
- Sebaliknya, jika kondisi yang diseleksi adalah tidak benar (bernilai logika = 0), maka pernyataan yang mengikutinya tidak akan diproses.

Mengenai kondisi harus ditulis diantara tanda kurung, sedangkan pernyataan dapat berupa sebuah pernyataan tunggal, pernyataan majemuk atau pernyataan kosong. Diagram alir dapat dilihat seperti Gambar 4.1.

Contoh penggunaan pernyataan if, misalkan untuk menentukan besarnya potongan harga yang diterima oleh seorang pembeli, berdasarkan kriteria:

- Tidak ada potongan harga jika total pembelian kurang dari Rp. 100.000 (dalam hal ini potongan harga diinisialisasi dengan nol).
- Bila total pembelian lebih dari atau sama dengan Rp. 100.000, potongan harga yang diterima dirubah menjadi sebesar 5% dari total pembelian.

```
/* File program : discount.c
Contoh penggunaan if untuk menghitung nilai discount */
#include <stdio.h>
main()
{
    double total_pembelian, discount = 0;
    /* discount diinisialisasi dengan nilai 0 */
    printf("Total pembelian = Rp ");
    scanf("%lf", &total_pembelian);
    if(total_pembelian >= 100.000)
        discount = 0.05 * total_pembelian;
    printf("Besarnya discount = Rp %.2lf\n",
discount);
}
```

Contoh eksekusi :

Total pembelian = Rp 200000
Besarnya discount = Rp 10000.00

Untuk pernyataan **if** yang diikuti dengan pernyataan majemuk, bentuknya adalah sebagai berikut :

```
if (kondisi)
{ /* tanda awal pernyataan majemuk*/
    pernyataan-1;
    pernyataan-2;
    .
    .
    .
    pernyataan-n;
} /* tanda akhir pernyataan majemuk */
```

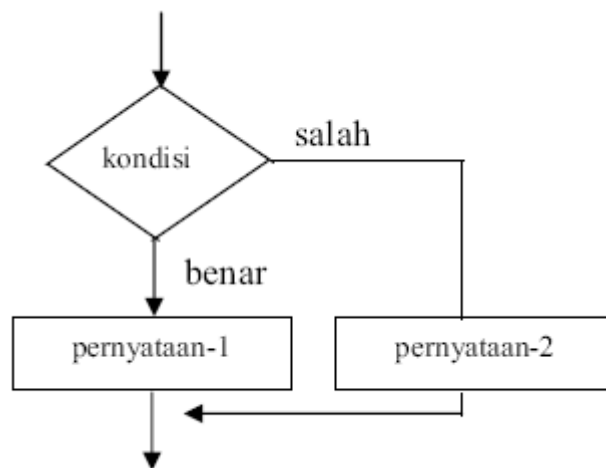
Pernyataan-pernyataan yang berada dalam tanda kurung { dan } akan dijalankan hanya bila kondisi *if* bernilai benar.

4.4 Pernyataan *if-else*

Pernyataan *if-else* memiliki bentuk :

```
if (kondisi)
    pernyataan-1;
else
    pernyataan-2;
```

Diagram alir dapat dilihat seperti gambar 4.2.



Gambar 4.2. Diagram alir *if-else*

Arti dari pernyataan *if-else* :

- Jika kondisi benar, maka **pernyataan-1** dijalankan.
- Sedangkan bila kondisi bernilai salah, maka **pernyataaan-2** yang dijalankan.

Masing-masing **pernyataan-1** dan **pernyataan-2** dapat berupa sebuah pernyataan tunggal, pernyataan majemuk ataupun pernyataan kosong.

Contoh penggunaan pernyataan *if-else*, misalkan untuk menyeleksi nilai suatu bilangan pembagi. Jika nilai bilangan pembagi adalah nol, maka hasil pembagian dengan nilai nol akan mendapatkan hasil tak berhingga. Jika ditemui nilai pembaginya nol, maka proses pembagian tidak akan dilakukan.

```

/* File program : bagi.c
Pemakaian if-else untuk menyeleksi bilangan pembagi */
#include <stdio.h>
main()
{
    float a, b;
    printf("Masukkan nilai a : ");
    scanf("%f", &a);
    printf("Masukkan nilai b : ");
    scanf("%f", &b);
    if (b == 0)
        printf("\n%g dibagi dengan nol = TAK BERHINGGA\n", a);
    else
        printf("\n%g dibagi dengan %g = %g\n", a, b, a/b);
}

```

Contoh eksekusi :

Masukkan nilai a : 5

Masukkan nilai b : 0

5 dibagi dengan nol = TAK BERHINGGA

4.5 Pernyataan *if* di dalam *if*

Di dalam suatu pernyataan *if* (atau *if-else*) bisa saja terdapat pernyataan *if* (atau *if-else*) yang lain. Bentuk seperti ini dinamakan sebagai *nested if*. Secara umum, bentuk dari pernyataan ini adalah sebagai berikut :

```

if (kondisi-1)
    if (kondisi-2)
        :
        .
        if(kondisi-n)
            pernyataan;
        else
            pernyataan;
        :
        .
    else
        pernyataan;
else
    pernyataan;

```


- Kondisi yang akan diseleksi pertama kali adalah kondisi yang terluar (**kondisi-1**). Jika **kondisi-1** bernilai salah, maka statemen *else* yang terluar (pasangan *if* yang bersangkutan) yang akan diproses. Jika *else* (pasangannya tsb) tidak ditulis, maka penyeleksian kondisi akan dihentikan.
- Jika **kondisi-1** bernilai benar, maka kondisi berikutnya yang lebih dalam (**kondisi-2**) akan diseleksi. Jika **kondisi-2** bernilai salah, maka statemen *else* pasangan dari *if* yang bersangkutan yang akan diproses. Jika *else* (untuk **kondisi-2**) tidak ditulis, maka penyeleksian kondisi akan dihentikan.
- Dengan cara yang sama, penyeleksian kondisi akan dilakukan sampai dengan **kondisi-n**, jika kondisi-kondisi sebelumnya bernilai benar.

```

/* File program : diskriminan1.c
Program untuk menghitung diskriminan dan akar-akar
persamaan kuadrat menggunakan if bersarang */
#include <stdio.h>
#include <math.h>
main()
{
    float a, b, c, d = 0;
    double x1, x2, imaginair;
    printf("MENCARI AKAR-AKAR PERSAMAAN KUADRAT
a+bx+c=0\n");
    printf("\nMasukkan nilai a : ");
    scanf("%f", &a);
    printf("Masukkan nilai b : ");
    scanf("%f", &b);
    printf("Masukkan nilai c : ");
    scanf("%f", &c);
    d = b*b-4*a*c; /* menghitung diskriminan */
    if (d >= 0)
        if (d == 0)
        {
            x1 = -b / (2 * a);
            printf("\nDua akar real kembar
yaitu : \n");
            printf("x1 = x2 = %g\n", x1);
        }
        else

```

```

        {
            x1 = (-b + sqrt(d))/(2*a);
            x2 = (-b - sqrt(d))/(2*a);
            printf("\nDua akar real berlainan
            yaitu :\n");
            printf("x1 = %g\n", x1);
            printf("x2 = %g\n", x2);
        }
    else
    {
        imaginair = (sqrt(-d)/(2*a));
        x1 = -b/(2*a);
        printf("\nDua akar imaginair berlainan
        yaitu : \n");
        printf("x1 = %g + %gi\n", x1,
        imaginair);
        printf("x2 = %g - %gi\n", x1,
        imaginair);
    }
}

```

Contoh eksekusi :

MENCARI AKAR-AKAR PERSAMAAN KUADRAT $a+bx+c=0$

Masukkan nilai a : 3

Masukkan nilai b : 6

Masukkan nilai c : 2

Dua akar real berlainan yaitu :

X1 = -0.42265

X2 = -1.57735

4.6 Pernyataan *else-if*

Contoh implementasi *nested if* ini misalnya pembuatan sebuah program kalkulator sederhana. User memberikan masukan dengan format :

operand1 operator operand2

Jenis operasi yang dikenakan bergantung pada jenis **operator** yang dimasukkan oleh user. Oleh karena itu program akan mengecek apakah **operator** berupa tanda '*', '/', '+', ataukah tanda '-'.

- Jika operator berupa tanda '*' maka **operand1** akan dikalikan dengan **operand2**
- Jika operator berupa tanda '/' maka **operand1** akan dibagi dengan **operand2**
- Jika operator berupa tanda '+' maka **operand1** akan dijumlahkan dengan **operand2**
- Jika operator berupa tanda '-' maka **operand1** akan dikurangi dengan **operand2**
- Kalau operator yang dimasukkan bukan merupakan salah satu dari jenis operator di atas, maka ekspresi tersebut tidak akan diproses, dan user akan mendapatkan pesan berupa : *"Invalid operator !"*

```

/* File program : kalkulator1.c
Contoh penggunaan else if untuk
mengimplementasikan program kalkulator sederhana
*/
#include <stdio.h>
main()
{
    int valid_operator = 1;
    /* valid_operator diinisialisasi dengan
logika 1 */
    char operator;
    float number1, number2, result;
    printf("Masukkan 2 buah bilangan dan sebuah
operator\n");
    printf("dengan format : number1 operator
number2\n\n");
    scanf("%f %c %f", &number1, &operator,
&number2);
    if(operator == '*')
        result = number1 * number2;
    else if(operator == '/')
        result = number1 / number2;
    else if(operator == '+')
        result = number1 + number2;
    else if(operator == '-')
        result = number1 - number2;

```

```

else
    valid_operator = 0;
if(valid_operator)
    printf("\n%g %c %g is %g\n", number1,
operator, number2, result );
else
    printf("Invalid operator!\n");
}

```

Contoh eksekusi :

Masukkan 2 buah bilangan dan sebuah operator

dengan format : number1 operator number2

23.2 + 12

23.2 + 12 is 35.2

4.7 Pernyataan *switch*

Pernyataan *switch* merupakan pernyataan yang dirancang khusus untuk menangani pengambilan keputusan yang melibatkan sejumlah alternatif, misalnya untuk menggantikan pernyataan *if* bertingkat.

Bentuk umum pernyataan *switch* adalah :

```

switch (ekspresi)
{
    case konstanta-1:
        pernyataan-1;
        .....
        break;
    case konstanta-2:
        .
        .
        .
    case konstanta-n:
        pernyataan-n;
        .....
        break;
    default:
        .....
        .....
        break;
}

```

dengan **ekspresi** dapat berupa ekspresi bertipe integer atau bertipe karakter. Demikian juga **konstanta-1**, **konstanta-2**, ..., **konstanta-n** dapat berupa konstanta integer atau karakter. Setiap pernyataan-i (**pernyataan-1**, ..., **pernyataan-n**) dapat berupa pernyataan tunggal ataupun pernyataan jamak. Dalam hal ini urutan penulisan pernyataan *case* tidak berpengaruh. Proses penyeleksian berlangsung sebagai berikut :

- Pengujian pada *switch* akan dimulai dari **konstanta-1**. Kalau nilai **konstanta-1** cocok dengan ekspresi maka **pernyataan-1** dijalankan. Kata kunci *break* harus disertakan di bagian akhir setiap pernyataan *case*, yang akan mengarahkan eksekusi ke akhir *switch*.
- Kalau ternyata **pernyataan-1** tidak sama dengan nilai **ekspresi**, pengujian dilanjutkan pada **konstanta-2**, dan berikutnya serupa dengan pengujian pada **konstanta-1**.
- Jika sampai pada pengujian *case* yang terakhir ternyata tidak ada kecocokan, maka pernyataan yang mengikuti kata kunci *default* yang akan dieksekusi. Kata kunci *default* ini bersifat opsional.
- Tanda kurung kurawal tutup (*)* menandakan akhir dari proses penyeleksian kondisi *case*.

Di bawah ini contoh program pemakaian pernyataan *switch* untuk menggantikan *if-else* bertingkat pada program **kalkulator1.c** di atas.

```
/* File program : kalkulator2.c
Contoh penggunaan pernyataan switch untuk
mengimplementasikan
kalkulator sederhana */
#include <stdio.h>
main()
{
    int valid_operator = 1;
    char operator;
    float number1, number2, result;
    printf("Masukkan 2 buah bilangan dan sebuah
operator\n");
    printf("dengan format : number1 operator
number2\n\n");
    scanf("%f %c %f", &number1, &operator, &number2);
    switch(operator) {
        case '*' : result = number1 * number2; break;
```

```

    case '/' : result = number1 / number2; break;
    case '+' : result = number1 + number2; break;
    case '-' : result = number1 - number2; break;
    default : valid_operator = 0;
}
if(valid_operator)
    printf("%g %c %g is %g\n", number1, operator,
    number2,result);
else
    printf("Invalid operator!\n");
}

```

Contoh eksekusi :

Masukkan 2 buah bilangan dan sebuah operator

Dengan format : number1 operator number2

23.2 = 12

invalid operator !

Kesimpulan :

- Operator kondisi adalah operator yang digunakan untuk menghasilkan kondisi benar (*true*) dan salah (*false*), yang terdiri atas operator relasi dan operator logika.
- Operator relasi biasa dipakai untuk membandingkan dua buah nilai.
- Operator logika biasa dipakai untuk menghubungkan ekspresi relasi.
- Untuk penyeleksian kondisi dalam rangka pengambilan keputusan bisa digunakan salah satu dari pernyataan berikut ini :

a. Pernyataan *if*, bentuk umumnya :

```

if (kondisi )
    pernyataan;

```

b. Pernyataan *if-else*, bentuk umumnya :

```

if (kondisi)
    pernyataan-1;
else
    pernyataan-2;

```

- c. Pernyataan *if* di dalam *if*, bentuk umumnya :

```
if (kondisi-1)
    if (kondisi-2)
        ...
        if(kondisi-n)
            pernyataan;
        else
            pernyataan;
        ...
    else
        pernyataan;
else
    pernyataan;
```

- d. Pernyataan *else-if*, bentuk umumnya :

```
if (kondisi-1)
    pernyataan-1;
else if (kondisi-2)
    pernyataan-2;
    ...
else if(kondisi-n)
    pernyataan-n;
else
    pernyataan-(n+1);
```

- e. Pernyataan *switch*, bentuk umumnya :

```
switch (ekspresi)
{
    case konstanta-1:
        pernyataan-1;
        .....
        break;
    case konstanta-2:
        .....
    case konstanta-n:
        pernyataan-n;
        .....
        break;
    default:
        .....
        break;
}
```

Latihan :

Buatlah potongan program untuk soal-soal di bawah ini

1. Gunakan statemen *if* untuk membandingkan nilai dari sebuah variabel integer (**sum**) dengan nilai 65. Jika lebih kecil, maka tampilkan pesan : *“Maaf, Anda harus mencoba lagi!”*.
2. Jika variabel **total** sama dengan variabel **tebak**, cetaklah nilai dari **total**, jika tidak sama, maka cetaklah nilai dari **tebak**.
3. Jika variabel **sum** sama dengan 10 dan variabel **total** kurang dari 20, maka tampilkan pesan : *“Tidak sesuai!”*
4. Jika variabel **flag** sama dengan 1 atau variabel **letter** bukan ‘X’, maka *assign* nilai 0 kepada variabel **exit_flag**, jika tidak, maka set **exit_flag** sama dengan 1.
5. Tulislah kembali pernyataan-pernyataan di bawah ini dengan menggunakan pernyataan *switch*

```
if( letter == 'X' )
    sum = 0;
else if ( letter == 'Z' )
    valid_flag = 1;
else if( letter == 'A' )
    sum = 1;
else
    printf("Unknown letter -->%c\n", letter );
```


BAB V

PENGULANGAN PROSES

5.1 Tujuan

1. Menjelaskan proses pengulangan menggunakan pernyataan *for*
2. Menjelaskan proses pengulangan menggunakan pernyataan *while*
3. Menjelaskan proses pengulangan menggunakan pernyataan *do-while*
4. Menjelaskan penggunaan pernyataan *break*
5. Menjelaskan penggunaan pernyataan *continue*
6. Menjelaskan penggunaan pernyataan *goto*
7. Menjelaskan loop di dalam loop (*nested loop*) dan contoh kasusnya
8. Menjelaskan penggunaan *exit()* untuk menghentikan eksekusi program dan contoh kasusnya

5.2 Pernyataan *for*

Mengulang suatu proses merupakan tindakan yang banyak dijumpai dalam pemrograman. Pada semua bahasa pemrograman, pengulangan proses ditangani dengan suatu mekanisme yang disebut *loop*. Dengan menggunakan *loop*, suatu proses yang berulang misalnya menampilkan tulisan yang sama seratus kali pada layar dapat diimplementasikan dengan kode program yang pendek.

Pernyataan pertama yang digunakan untuk keperluan pengulangan proses adalah pernyataan *for*. Bentuk pernyataan ini :

```
for (ungkapan1; ungkapan2; ungkapan3)
    pernyataan;
```

Kegunaan dari masing-masing ungkapan pada pernyataan *for*.

- Ungkapan1 : digunakan untuk memberikan inisialisasi terhadap variabel pengendali *loop*.
- Ungkapan2 : dipakai sebagai kondisi untuk keluar dari *loop*.
- Ungkapan3 : dipakai sebagai pengatur kenaikan nilai variabel pengendali *loop*.

Ketiga ungkapan dalam *for* tersebut harus dipisahkan dengan tanda titik koma (;). Dalam hal ini pernyataan bisa berupa pernyataan tunggal maupun jamak. Jika pernyataannya berbentuk jamak, maka pernyataan-pernyataan tersebut harus diletakkan di antara kurung kurawal buka ({) dan kurung kurawal tutup (}), sehingga formatnya menjadi :

```
for (ungkapan1; ungkapan2; ungkapan3)
{
    pernyataan;
    pernyataan;
    .
}
```

Contoh penggunaan *for*, misalnya untuk menampilkan deretan angka sebagai berikut :

```
20
30
40
50
.
100
```

Untuk keperluan ini, pernyataan *for* yang digunakan berupa :

```
for (bilangan = 20; bilangan <= 100; bilangan += 10)
    printf("%d\n", bilangan);
```

```
/* File program : for1.c
```

```
Contoh pemakaian for untuk membentuk deret naik */
```

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    int bilangan;
```

```
    for(bilangan = 20; bilangan <= 100; bilangan +=
        10) printf("%d\n", bilangan);
```

```
}
```

Contoh eksekusi :

```
20
30
40
50
```

60
70
80
90
100

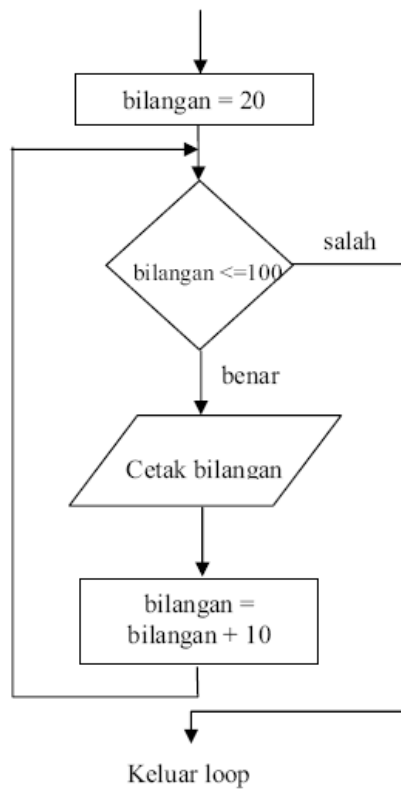
Pada program di atas, kenaikan terhadap variabel pengendali *loop* sebesar 10 (positif), yang dinyatakan dengan ungkapan

`bilangan += 10`

yang sama artinya dengan

`bilangan = bilangan + 10`

Kalau digambarkan dalam bentuk diagram alir, akan terlihat sbb :



Gambar 5.1. Diagram alir *for*

Pada contoh yang melibatkan pernyataan *for* di atas, kenaikan variabel pengendali *loop* berupa nilai positif. Sebaliknya kenaikan terhadap variabel pengendali *loop* bisa diatur bernilai negatif. Cara ini dapat digunakan untuk memperoleh deret sebagai berikut.

```
60
50
40
30
20
10
```

Untuk itu selengkapnya program yang dibutuhkan seperti berikut.

```
/* File program : for2.c
Contoh pemakaian for untuk membentuk deret turun */
#include <stdio.h>
main()
{
    int bilangan;
    for (bilangan = 60; bilangan >= 10; bilangan -=
10)
        printf("%d\n", bilangan);
}
```

Contoh eksekusi :

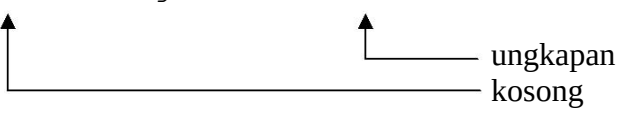
```
60
50
40
30
20
10
```

Kadang-kadang dijumpai adanya pernyataan *for* yang tidak mengandung bagian ungkapan yang lengkap (beberapa ungkapan dikosongkan). Dengan cara ini, pernyataan

```
for (bilangan = 20; bilangan <= 100; bilangan += 10)
    printf("%d\n", bilangan);
```

dapat ditulis menjadi:

```
bilangan = 20; /* inisialisasi di luar for */
for (    ; bilangan <= 100;    )
{
    printf("%d\n", bilangan);
    bilangan += 10;
}
```



Tampak bahwa ungkapan yang biasa dipakai untuk inisialisasi variabel pengendali *loop* tak ada. Sebagai gantinya pengendalian *loop* diatur sebelum pernyataan *for*, berupa

```
bilangan = 20;
```

Pengosongan ini juga dilakukan pada ungkapan yang biasa dipakai untuk menaikkan nilai variabel pengendali *loop*. Sebagai gantinya, di dalam tubuh *loop* diberikan pernyataan untuk menaikkan nilai variabel pengendali *loop*, yaitu berupa

```
bilangan += 10;
```

Ungkapan yang tidak dihilangkan berupa `bilangan <=100`. Ungkapan ini tetap disertakan karena dipakai sebagai kondisi untuk keluar dari *loop*.

Sesungguhnya ungkapan yang dipakai sebagai kondisi keluar dari *loop* juga bisa dihilangkan, sehingga bentuknya menjadi

```
for (;;)
    pernyataan
```

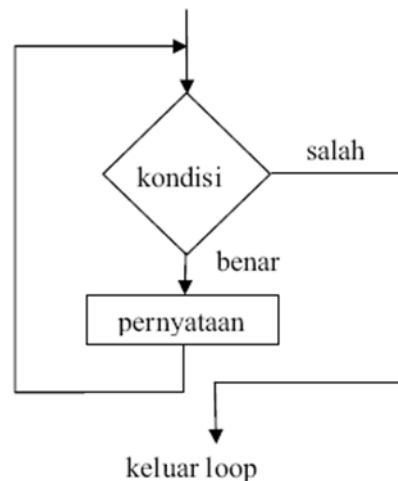
Suatu pertanyaan mungkin timbul “Lalu bagaimana caranya kalau ingin keluar dari *loop* pada bentuk di atas?”. Caranya adalah dengan menggunakan pernyataan yang dirancang khusus untuk keluar dari *loop*. Mengenai hal ini akan dibahas pada sub bab yang lain.

5.3 Pernyataan *while*

Pada pernyataan *while*, pengecekan terhadap *loop* dilakukan di bagian awal (sebelum tubuh *loop*). Lebih jelasnya, bentuk pernyataan *while* adalah sebagai berikut.

<pre>while (kondisi) pernyataan;</pre>
--

Pernyataan dapat berupa pernyataan tunggal, pernyataan majemuk ataupun pernyataan kosong. Proses pengulangan terhadap pernyataan dijelaskan pada Gambar 5.2. Pada gambar tersebut, tampak bahwa ada kemungkinan pernyataan yang merupakan tubuh *loop* tidak dijalankan sama sekali, yaitu kalau hasil pengujian kondisi *while* yang pertama kali ternyata bernilai salah.



Gambar 5.2. Diagram alir *while*

Contoh pemakaian *while* misalnya untuk mengatur agar tombol yang ditekan oleh pemakai program berupa salah satu diantara 'Y','y', 'T' atau 't'.

Impelementasinya :

```

/*File program : pilihan.c
Untuk membaca tombol Y atau T */
#include <stdio.h>
main()
{
    char pilihan;
    /* diberi nilai salah lebih dahulu */
    int sudah_benar = 0;
    printf("Pilihlah Y atau T.\n");
    /*program dilanjutkan jika tombol Y,y,T atau t
    ditekan*/
    while(!sudah_benar)
    {
        pilihan = getchar();    /* baca tombol */
        sudah_benar = (pilihan == 'Y') || (pilihan
            == 'y') || (pilihan == 'T') ||
            (pilihan == 't');
    }
    /* memberi keterangan tentang pilihan */
    switch(pilihan)
    {
        case 'Y': case 'y':
            puts("\nPilihan anda adalah Y"); break;
        case 'T': case 't':

```

```

        puts("\nPilihan anda adalah T");
    }
}

```

Contoh eksekusi :

Pilihlah Y atau T

Pilihan anda adalah Y

Inisialisasi terhadap variabel `sudah_benar` yang akan dijalankan pada kondisi *while* dengan memberi nilai awal bernilai *false* (`sudah_benar = 0`) dimaksudkan agar tubuh *loop*

```

{
    pilihan = getchar( ); /* baca tombol */
    sudah_benar = (pilihan == 'Y') || (pilihan ==
        'y') || (pilihan == 'T') || (pilihan == 't');
}

```

dijalankan minimal sekali.

Contoh lain pemakaian *while* dapat dilihat pada program yang digunakan untuk menghitung banyaknya karakter dari kalimat yang dimasukkan melalui keyboard (termasuk karakter spasi). Untuk mengakhiri pemasukan kalimat, tombol ENTER (`'\n'`) harus ditekan. Karena itu, tombol ENTER inilah yang dijadikan kondisi penghitungan jumlah spasi maupun karakter seluruhnya. Lengkapnya, kondisi yang dipakai dalam *while* berupa :

```
while((kar = getchar()) != '\n')
```

Ungkapan di atas mempunyai arti :

- Bacalah sebuah karakter dan berikan ke variabel **kar**
- Kemudian bandingkan apakah karakter tersebut = `'\n'` (ENTER)

Ungkapan menghasilkan nilai benar jika tombol yang ditekan bukan ENTER.

Pada program, kalau tombol yang ditekan bukan ENTER , maka :

- Jumlah karakter dinaikkan sebesar satu melalui pernyataan : `jumkar++`;
- Kalau karakter berupa SPASI, maka jumlah spasi dinaikkan sebesar satu, melalui pernyataan : `if (kar == ' ') jumspasi++`;

```

/* File program : jumkar.c
Menghitung jumlah kata dan karakter dalam suatu
kalimat */
#include <stdio.h>
main()
{
    char kar;
    int jumkar = 0, jumspasi = 0;
    puts("Masukkan sebuah kalimat dan akhiri dgn
ENTER.\n"); puts("Saya akan menghitung jumlah
karakter "); puts("pada kalimat tersebut.\n");
    while((kar = getchar()) != '\n')
    {
        jumkar++;
        if (kar == ' ') jumspasi++;
    }
    printf("\nJumlah karakter      = %d", jumkar);
    printf("\nJumlah SPASI      = %d\n\n", jumspasi);
}

```

Contoh eksekusi :

Masukkan sebuah kalimat, akhiri dgn ENTER.

Saya akan menghitung jumlah karakter pada kalimat tersebut.

Belajar bahasa C sangat menyenangkan

Jumlah karakter = 36

Jumlah SPASI = 4

5.4 Pernyataan *do-while*

Bentuk pernyataan *do-while*

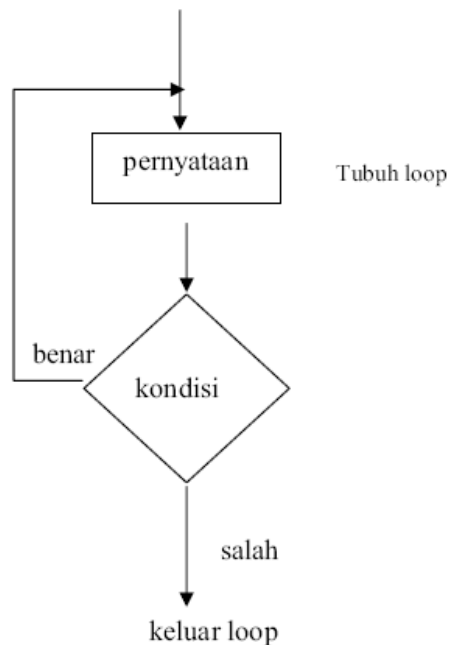
```

do
    pernyataan;
while (kondisi)

```

Pada pernyataan *do-while*, tubuh *loop* berupa pernyataan, dengan pernyataan bisa berupa pernyataan tunggal, pernyataan majemuk ataupun pernyataan kosong. Pada pernyataan *do*, mula-mula pernyataan dijalankan. Selanjutnya, kondisi diuji. Sendainya kondisi bernilai benar, maka pernyataan dijalankan lagi, kemudian kondisi diperiksa kembali, dan seterusnya. Kalau

kondisi bernilai salah pada saat dites, maka pernyataan tidak dijalankan lagi. Untuk lebih jelasnya dapat dilihat pada Gambar 5.3. Berdasarkan Gambar 5.3 terlihat bahwa tubuh *loop* minimal akan dijalankan sekali.



Gambar 5.3. Diagram alir do-while

Program berikut memberikan contoh pemakaian do-while untuk mengatur penampilan tulisan "BAHASA C" sebanyak sepuluh kali.

Contoh:

```
i = 0;
do
{
    puts("BAHASA C");
    i++;
} while(i<10);
```

Pada program di atas, variabel pencacah dipakai untuk menghitung jumlah tulisan yang sudah ditampilkan pada layar. Selama nilai pencacah kurang dari 10, maka perintah

```
puts("BAHASA C");
```

akan dilaksanakan kembali

Penanganan pembacaan tombol pada contoh program **pilihan.c** yang memakai *while* di atas, kalau diimplementasikan dengan memakai *do-while* dapat dibuat seperti berikut.

```
/* File program : pilihan2.c
Untuk membaca tombol Y atau T */
#include <stdio.h>
main()
{
    char pilihan;
    int sudah_benar;
    printf("Pilihlah Y atau T.\n");
    /* program dilanjutkan kalau tombol Y,y,T atau t
    ditekan */
    do
    {
        pilihan = getchar( ); /* baca tombol */
        sudah_benar = (pilihan == 'Y') || (pilihan ==
        'y') || (pilihan == 'T') || (pilihan == 't');
    } while(! sudah_benar);

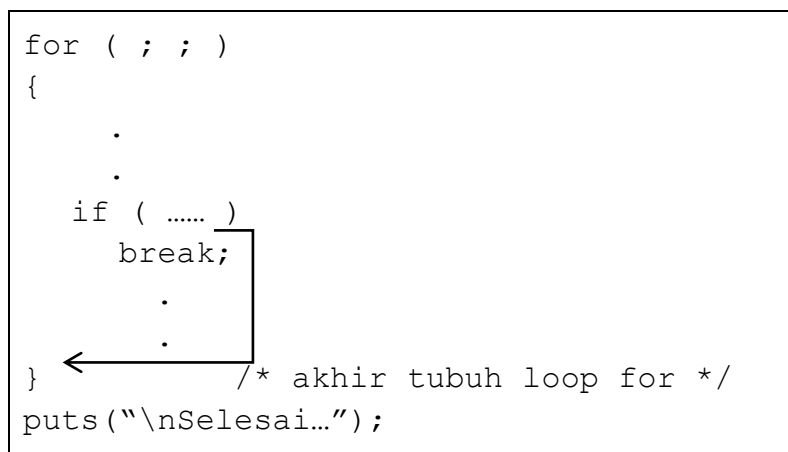
    /* memberi keterangan tentang pilihan */
    switch(pilihan)
    {
        case 'Y':
        case 'y':
            puts("\nPilihan anda adalah Y");
            break;
        case 'T':
        case 't':
            puts("\nPilihan anda adalah T");
    }
}
```

Mula-mula tombol dibaca dengan menggunakan *getchar()* dan kemudian diberikan ke variabel *pilihan*. Sesudah itu, variabel **sudah_benar** akan diisi dengan nilai benar (1) atau salah (0) tergantung dari nilai *pilihan*. Kalau *pilihan* berisi salah satu diantara ‘Y’, ‘y’, ‘T’ atau ‘t’, maka sudah berisi salah satu diantara ‘Y’, ‘y’, ‘T’ atau ‘t’, maka **sudah_benar** akan berisi benar. Nilai pada

variabel **sudah_benar** ini selanjutnya dijadikan sebagai kondisi *do-while*. Pengulangan terhadap pembacaan tombol akan dilakukan kembali selama **sudah_benar** bernilai salah.

5.5 Pernyataan *break*

Pernyataan *break* sesungguhnya telah diperkenalkan pada pernyataan *switch*. Pernyataan ini berfungsi untuk keluar dari *loop for*, *do-while* dan *while*. Sedangkan pada *switch* yaitu untuk menuju ke akhir (keluar dari) struktur *switch*. Sebagai contoh dapat dilihat pada gambar 5.4. Kalau pernyataan *break* dijalankan maka eksekusi akan dilanjutkan ke pernyataan yang terletak sesudah akhir tubuh *loop for*.



Gambar 5.4 Ilustrasi pengaruh *break*

Pada contoh potongan program berikut, pembacaan dan penampilan terhadap tombol yang ditekan akan berakhir kalau tombol yang ditekan adalah ENTER (‘\n’). Pernyataan yang digunakan untuk keperluan ini :

```
if (kar == '\n')
    break; /* keluar dari loop for */
```

Potongan program ini menyatakan “Jika tombol yang ditekan berupa ENTER, maka keluarlah dari *loop for*”. Untuk lebih jelasnya, perhatikan program berikut.

```
/* File program : tamat.c
Pemakaian break untuk keluar dari looping */
#include <stdio.h>
```

```

main()
{
    char kar;
    printf("Ketik sembarang kalimat");
    printf(" dan akhiri dengan ENTER\n\n");
    for ( ; ; )
    {
        kar = getchar(); if(kar == '\n') break;
    }
    printf("Selesai\n");
}

```

Contoh eksekusi :

Ketik sembarang kalimat dan akhiri dengan ENTER :

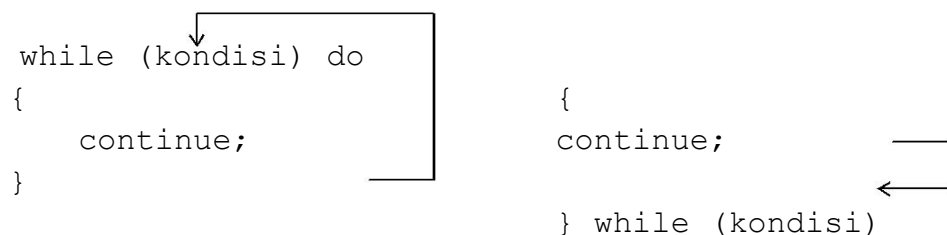
Menulis apa saja

Selesai

Jika pernyataan *break* berada dalam *loop* yang bertingkat (*nested loop*), maka pernyataan *break* hanya akan membuat proses keluar dari *loop* yang bersangkutan (tempat *break* dituliskan), bukan keluar dari semua *loop*.

5.6 Pernyataan *Continue*

Pernyataan *continue* digunakan untuk mengarahkan eksekusi ke iterasi (proses) berikutnya pada *loop* yang sama. Pada *do-while* dan *while*, pernyataan *continue* menyebabkan eksekusi menuju ke kondisi pengujian pengulangan, seperti yang dilukiskan pada Gambar 5.5. Pada *loop for*, pernyataan *continue* menyebabkan bagian penaik variabel pengendali *loop* dikerjakan (ungkapan3 pada struktur *for*) dan kondisi untuk keluar dari *loop for* (ungkapan2 pada struktur *for*) diuji kembali.



Gambar 5.5 Pengaruh *continue* pada *while* dan *do-while*

Program ini dapat digunakan untuk memasukkan data yang harus diulangi dan hal ini dikendalikan dengan *continue*. Untuk mengakhiri pemasukan data, data yang dimasukkan harus bernilai 0. Perlu diketahui kondisi bernilai 1, menyatakan bahwa kondisi selalu dianggap benar. Untuk keluar dari *loop*, pernyataan yang digunakan berupa *break*.

Pengaruh *continue* pada *loop for* diperlihatkan pada program berikut ini. Program ini dipakai untuk menampilkan bilangan ganjil yang terletak antara 7 sampai dengan 25, kecuali 15.

```
/* File program : ganjil.c
menampilkan bilangan ganjil antara 7 - 25 kecuali 15
*/
#include <stdio.h>
main()
{
    int x;
    for (x = 7; x <= 25; x += 2)
    {
        if (x == 15)
            continue;
        printf("%4d", x);
    }
    printf("\n");
}
```

Contoh eksekusi :

9 11 13 17 19 21 23 25

Pada program di atas, untuk menghindari agar nilai 15 tidak ditampilkan ke layar, pernyataan yang digunakan berupa

```
if (x == 15)
    continue;
```

Artinya, jika kondisi `x == 15` bernilai benar, pernyataan *continue* menyebabkan pernyataan sisanya yaitu

```
printf("%d", x);
```

diabaikan dan eksekusi diarahkan kepada ungkapan :

```
x += 2
```

dan kemudian menguji kondisi :

```
x <= 25
```

Pada program di atas, pernyataan :

```
for (x = 7; x <= 25; x += 2)
{
    if (x == 15)
        continue;
    printf("%4d", x);
}
```

dapat ditulis dalam bentuk lain sebagai berikut :

```
for (x = 7; x <= 25; x += 2)
    if (x != 15)
        printf("%4d", x);
```

5.7 Loop Di Dalam Loop

Dalam suatu *loop* bisa terkandung *loop* yang lain. *Loop* yang terletak di dalam *loop* biasa disebut dengan *loop* di dalam *loop* (*nested loop*). Salah satu contoh *nested loop* misalnya pada permasalahan untuk membuat tabel perkalian:

	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	2	4	6	8	10	12	14	16
3	3	6	9	12	15	18	21	24
4	4	8	12	16	20	24	28	32
5	5	10	15	20	25	30	35	40
6	6	12	18	24	30	36	42	48
7	7	14	21	28	35	42	49	56
8	8	16	24	32	40	48	56	64

Implementasi dalam program selengkapnya adalah sebagai berikut :

```
/* File program : tblkali.c
Loop for bersarang untuk membuat tabel perkalian */
#include <stdio.h>
#define MAKS 8
```

```

main()
{
    int baris, kolom, hasil_kali;
    for (baris = 1; baris <= MAKS; baris++)
    {
        for (kolom = 1; kolom <= MAKS; kolom++)
        {
            hasil_kali = baris * kolom; printf
            ("%2d", hasil_kali);
        }
        printf("\n"); /* pindah baris */
    }
}

```

Bagian yang terletak dalam bingkai di depan dapat diperoleh melalui

```

for (baris = 1; baris <= MAKS; baris++)
{
    hasil_kali = baris * kolom;
    printf ("%2d", hasil_kali);
}

```

dengan **MAKS** didefinisikan bernilai 8. Bagian *loop* yang terdalam :

```

for (kolom = 1; kolom <= MAKS; kolom++)
{
    hasil_kali = baris * kolom;
    printf ("%2d", hasil_kali);
}

```

digunakan untuk mencetak suatu deret hasil perkalian dalam satu baris. Untuk berpindah ke baris berikutnya, pernyataan yang digunakan yaitu

```
printf("\n");
```

Adapun pencetakan untuk semua baris dikendalikan melalui

```
for (baris = 1; baris <= MAKS; baris++)
```

Pernyataan di atas mempunyai arti “dari baris ke-1 sampai dengan baris ke-MAKS”.

5.8 Pernyataan *goto*

Pernyataan *goto* merupakan intruksi untuk mengarahkan eksekusi ke pernyataan yang diawali dengan suatu label. Label sendiri berupa suatu pengenal (*identifier*) yang diikuti dengan tanda titik dua (:)

Contoh pemakaian *goto* ditujukan pada program dibawah ini:

Pernyataan

```
goto cetak;
```

Mengisyaratkan agar eksekusi dilanjutkan ke pernyataan yang diawali dengan label

```
cetak:
```

Pernyataan

```
if (++pencacah <= 10)
    goto cetak;
```

Mempunyai arti :

- Naikkan nilai **pencacah** sebesar 1
- Kemudian, jika **pencacah** kurang dari atau sama dengan 10 maka eksekusi menuju ke label **cetak**.

Penerapan *goto* biasanya dilakukan pada *loop* di dalam *loop* (*nested loop*), dengan tujuan memudahkan untuk keluar dari *loop* terdalam menuju ke pernyataan yang terletak di luar *loop* terluar.

5.9 Menggunakan *exit()* Untuk Menghentikan Eksekusi Program

Suatu eksekusi program dapat dihentikan (secara normal) melalui pemanggilan fungsi *exit()*. Hal ini biasa dilakukan, jika di dalam suatu eksekusi terdapat suatu kondisi yang tak dikehendaki. Prototipe dari fungsi *exit()* didefinisikan pada file **stdlib.h**, yang memiliki deklarasi sebagai berikut :

```
void exit(int status);
```

Menurut kebiasaan, nilai nol diberikan pada argumen *exit()* untuk menunjukkan penghentian program yang normal. Sedangkan untuk menunjukkan kesalahan, nilai yang diberikan pada argumen fungsi diisi dengan nilai bukan-nol. Pada contoh program berikut, eksekusi program akan dihentikan hanya jika tombol 'X' ditekan.


```

/* File program : keluar.c
Pemakaian exit() untuk menghentikan eksekusi program
*/
#include <stdio.h> #include <stdlib.h>
main()
{
    char kar;
    printf("Tekanlah X untuk menghentikan program.\n");
    for ( ; ;)
    {
        while ((kar = getchar()) == 'X') exit(0);
    }
}

```

Kesimpulan :

- Pada semua bahasa pemrograman, pengulangan proses ditangani dengan suatu mekanisme yang disebut *loop*.
- Pernyataan-pernyataan yang bisa digunakan untuk keperluan pengulangan proses (*looping*) adalah :

(a) Pernyataan *for*, dengan bentuk umum sebagai berikut:

```

for (ungkapan1; ungkapan2; ungkapan3)
    pernyataan;

```

Jika pernyataannya berbentuk jamak, maka pernyataan-pernyataan tersebut harus diletakkan di antara kurung kurawal buka ({) dan kurung kurawal tutup (})

(b) Pernyataan *while*, dengan bentuk umum sebagai berikut:

```

while (kondisi)
    pernyataan;

```

(c) Pernyataan *do-while*, dengan bentuk umum sebagai berikut :

```

do
    pernyataan;
while (kondisi)

```

- Pernyataan *break* berfungsi untuk keluar dari *loop for*, *do-while* dan *while*.
- Pernyataan *continue* digunakan untuk mengarahkan eksekusi ke iterasi (proses) berikutnya pada *loop* yang sama.
- Dalam suatu *loop* bisa terkandung *loop* yang lain (*nested loop*).
- Pernyataan *goto* merupakan intruksi untuk mengarahkan eksekusi ke pernyataan yang diawali dengan suatu label. Label sendiri berupa suatu pengenalan (*identifier*) yang diikuti dengan tanda titik dua (:).
- Suatu eksekusi program dapat dihentikan (secara normal) melalui pemanggilan fungsi *exit()*. Hal ini biasa dilakukan, jika di dalam suatu eksekusi terdapat suatu kondisi yang tak dikehendaki.

Latihan :

Buatlah potongan program untuk soal-soal di bawah ini

1. Gunakan *loop for* untuk menampilkan nilai 1 sampai dengan 10 dalam baris-baris yang terpisah.
2. Gunakan *loop for* untuk mendapatkan tampilan seperti berikut.


```

1
22
333
4444
55555
```
3. Gunakan *loop for* untuk menjumlahkan seluruh bilangan antara 10 sampai dengan 100 ke dalam sebuah variabel total. Asumsikan bahwa variabel total tidak diinisialisasi terlebih dahulu dengan nilai nol.
4. Gunakan *loop for* untuk menampilkan seluruh karakter dari A sampai dengan Z dalam baris-baris yang terpisah.
5. Hitunglah bilangan triangular dari masukan pengguna, yang dibaca dari keyboard dengan menggunakan *scanf()*. Bilangan triangular adalah penjumlahan dari bilangan masukan dengan seluruh bilangan sebelumnya, sehingga bilangan triangular dari 7 adalah : $7 + 6 + 5 + 4 + 3 + 2 + 1$

6. Gunakan *loop while* untuk menampilkan bilangan integer antara 1 sampai dengan 10 di layar sbb : 123456768910
7. Gunakan *nested while loop* untuk mendapatkan keluran seperti berikut
1
22
333
4444
55555

BAB VI

FUNGSI

6.1 Tujuan

1. Memecah program dalam fungsi fungsi yang sederhana.
2. Menjelaskan tentang pemrograman terstruktur.
3. Mengetahui perbedaan antara variabel lokal, eksternal, statis dan register

6.2 Dasar Fungsi

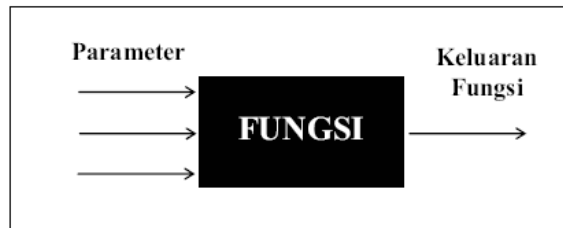
Fungsi adalah suatu bagian dari program yang dirancang untuk melaksanakan tugas tertentu dan letaknya dipisahkan dari program yang menggunakannya. Elemen utama dari program bahasa C berupa fungsi-fungsi, dalam hal ini program dari bahasa C dibentuk dari kumpulan fungsi pustaka (standar) dan fungsi yang dibuat sendiri oleh pemrogram. Fungsi banyak digunakan pada program C dengan tujuan :

- a. Program menjadi terstruktur, sehingga mudah dipahami dan mudah dikembangkan. Dengan memisahkan langkah-langkah detail ke satu atau lebih fungsi-fungsi, maka fungsi utama (*main()*) menjadi lebih pendek, jelas dan mudah dimengerti.
- b. dapat mengurangi pengulangan (duplikasi) kode. Langkah-langkah program yang sama dan dipakai berulang-ulang di program dapat dituliskan sekali saja secara terpisah dalam bentuk fungsi-fungsi. Selanjutnya bagian program yang membutuhkan langkah-langkah ini tidak perlu selalu menuliskannya, tetapi cukup memanggil fungsi-fungsi tersebut.

Fungsi standar C yang mengemban tugas khusus contohnya adalah ;

- *printf()* , yaitu untuk menampilkan informasi atau data ke layar.
- *scanf()* , yaitu untuk membaca kode tombol yang diinputkan.

Pada umumnya fungsi memerlukan nilai masukan atau parameter yang disebut sebagai argumen. Nilai masukan ini akan diolah oleh fungsi. Hasil akhir fungsi berupa sebuah nilai (disebut sebagai *return value* atau nilai keluaran fungsi). Oleh karena itu fungsi sering digambarkan sebagai "kotak gelap" seperti ditunjukkan pada Gambar 6.1



Gambar 6.1 Fungsi sebagai sebuah kotak gelap

Penggambaran sebagai kotak gelap di antaranya menjelaskan bahwa bagian dalam fungsi bersifat pribadi bagi fungsi. Tak ada suatu pernyataan di luar fungsi yang bisa mengakses bagian dalam fungsi, selain melalui parameter (atau variabel eksternal yang akan dibahas belakangan). Misalnya melakukan *goto* dari pernyataan di luar fungsi ke pernyataan dalam fungsi adalah tidak diperkenankan.

Bentuk umum dari definisi sebuah fungsi adalah sebagai berikut ;

```
tipe-keluaran-fungsi nama-fungsi (deklarasi argumen)
{
    tubuh fungsi
}
```

Keterangan :

- **tipe-keluaran-fungsi**, dapat berupa salah satu tipe data C, misalnya *char* atau *int*. Kalau penentu tipe tidak disebutkan maka dianggap bertipe *int* (secara *default*).
- **tubuh fungsi** berisi deklarasi variabel (kalau ada) dan statemen-statement yang akan melakukan tugas yang akan diberikan kepada fungsi yang bersangkutan. Tubuh fungsi ini ditulis di dalam tanda kurung kurawal buka dan kurung kurawal tutup.

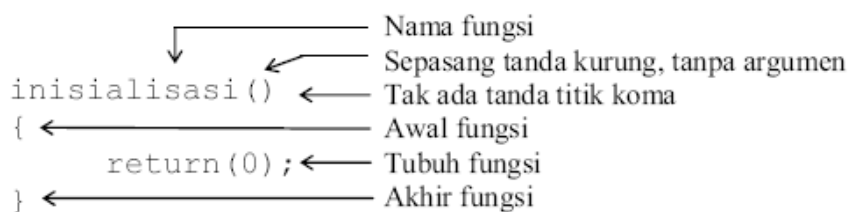
Sebuah fungsi yang sederhana bisa saja tidak mengandung parameter sama sekali dan tentu saja untuk keadaan ini deklarasi parameter juga tidak ada.

Contoh ;

```
int inisialisasi()
{
    return(0);
}
inisialisasi()
{
    return(0);
}
```

Pada fungsi di atas :

- tipe keluaran fungsi tidak disebutkan, berarti keluaran fungsi ber tipe int.
- *inisialisasi* adalah nama fungsi
- Tanda () sesudah nama fungsi menyatakan bahwa fungsi tak memiliki parameter.
- Tanda { dan } adalah awal dan akhir fungsi
- *return(0)* merupakan sebuah pernyataan dalam tubuh fungsi.



Gambar 6.2 Penjelasan definisi sebuah fungsi

6.3 Memberikan Nilai Keluaran Fungsi

Suatu fungsi dibuat untuk maksud menyelesaikan tugas tertentu. Suatu fungsi dapat hanya melakukan suatu tugas saja tanpa memberikan suatu hasil keluaran atau melakukan suatu tugas dan kemudian memberikan hasil keluaran. Fungsi yang hanya melakukan suatu tugas saja tanpa memberikan hasil keluaran misalnya adalah fungsi untuk menampilkan hasil di layar.

Dalam tubuh fungsi, pernyataan yang digunakan untuk memberikan nilai keluaran fungsi berupa *return*. Sebagai contoh, pada fungsi **inisialisasi()** di atas terdapat pernyataan

```
return(0);
```

merupakan pernyataan untuk memberikan nilai keluaran fungsi berupa nol. Selengkapnya perhatikan program berikut ini.

```
/* File program : inisial.c
Contoh pembuatan fungsi */
int inisialisasi();
#include <stdio.h>
main()
{
    int x, y;
```

```

    x = inisialisasi();
    printf("x = %d\n", x);
    y = inisialisasi();
    printf("y = %d\n", y);
}

int inisialisasi()
{
    return(0);
}

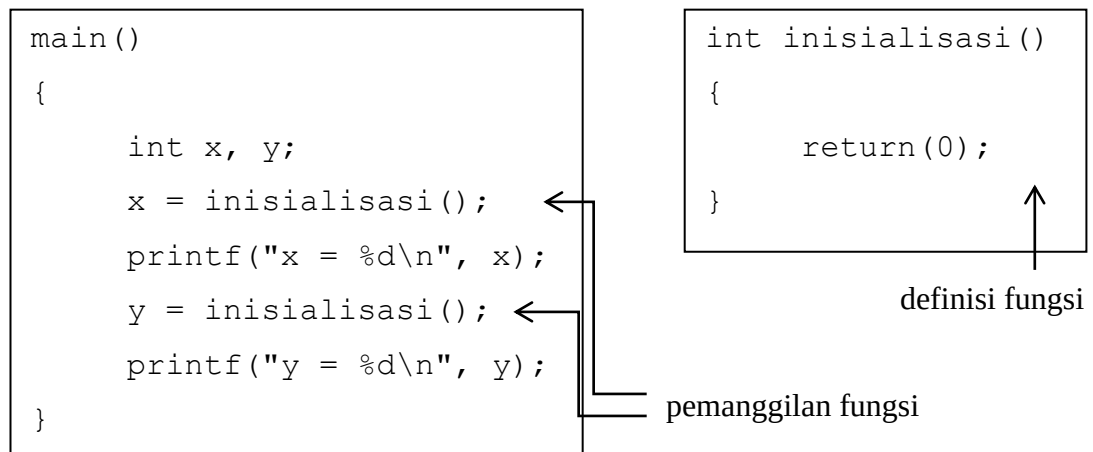
```

Contoh eksekusi :

x = 0

y = 0

Program menjelaskan bahwa suatu fungsi cukup didefinisikan satu kali tetapi bisa digunakan beberapa kali. Pada keadaan semacam ini seandainya tubuh fungsi mengandung banyak pernyataan, maka pemakaian fungsi dapat menghindari duplikasi kode dan tentu saja menghemat penulisan program maupun kode dalam memori.



Gambar 6.3 Proses pemanggilan fungsi

Misalnya pada saat pernyataan

```
x = inisialisasi();
```

dijalankan, mula-mula eksekusi akan diarahkan ke fungsi **inisialisasi()**, selanjutnya suatu nilai keluaran (hasil fungsi) akhir fungsi diberikan ke x. Proses yang serupa, dilakukan untuk pernyataan

```
y = inisialisasi();
```

Bagi suatu fungsi, jika suatu pernyataan *return* dieksekusi, maka eksekusi terhadap fungsi akan berakhir dan nilai pada parameter *return* akan menjadi keluaran fungsi. Untuk fungsi yang tidak memiliki pernyataan *return*, tanda `}` pada bagian akhir fungsi akan menyatakan akhir eksekusi fungsi.

Di bawah ini diberikan contoh sebuah fungsi yang mengandung dua buah pernyataan *return*. Fungsi digunakan untuk memperoleh nilai minimum di antara 2 buah nilai yang menjadi parameternya.

```
int minimum(int x, int y)
{
    if (x < y)
        return(x);
    else
        return(y);
}
```

Pada fungsi di atas terdapat dua buah parameter berupa *x* dan *y*. Oleh karena itu fungsi juga mengandung bagian untuk mendeklarasikan parameter, yang menyatakan *x* dan *y* bertipe *int*. Adapun penentuan nilai keluaran fungsi dilakukan pada tubuh fungsi, berupa pernyataan

```
if (x < y)
    return(x);
else
    return(y);
```

yang menyatakan :

- jika $x < y$ maka nilai keluaran fungsi adalah sebesar nilai *x*.
- untuk keadaan lainnya ($x \geq y$) maka keluaran fungsi adalah sebesar *y*.

Selengkapnya perhatikan program di bawah ini.

```
/* File program : minimum1.c */
#include <stdio.h>
int minimum (int, int);
main()
{
    int a, b, kecil;
    printf("Masukkan nilai a : "); scanf("%d", &a);
    printf("Masukkan nilai b : "); scanf("%d", &b);
    kecil = minimum(a, b);
```



```

        printf("\nBilangan terkecil antara %d dan %d
        adalah %d\n\n", a, b, kecil);
    }
    minimum(int x, int y)
    {
        if (x < y)
            return(x);
        else
            return(y);
    }

```

Contoh eksekusi :

Masukkan nilai a = 4

Masukkan nilai b = 2

Bilangan terkecil antara 4 dan 2 adalah 2

6.4 Fungsi Dengan Keluaran Bukan Integer

Untuk fungsi yang mempunyai keluaran bertipe bukan integer, maka fungsi haruslah didefinisikan dengan diawali tipe keluaran fungsinya (ditulis di depan nama fungsi). Sebagai contoh untuk menghasilkan nilai terkecil di antara dua buah nilai real, maka definisinya berupa :

```

float minimum(float x, float y)
{
    if (x < y)
        return(x);
    else
        return(y);
}

```

Perhatikan, di depan nama **minimum** diberikan tipe keluaran fungsi berupa *float*. Seluruh parameter sendiri juga didefinisikan dengan tipe *float*. Selengkapnya adalah sebagai berikut :

```

/* File program : minimum2.c */
#include <stdio.h>
float minimum (float, float);
main()
{

```

```

float a, b, kecil;
printf("Masukkan nilai a : "); scanf("%f", &a);
printf("Masukkan nilai b : "); scanf("%f", &b);
kecil = minimum(a, b);
printf("\nBilangan terkecil antara %g dan %g
adalah %g\n\n", a, b, kecil);
}
float minimum(float x, float y)
{
    if (x < y)
        return(x);
    else
        return(y);
}

```

Contoh eksekusi :

Masukkan nilai a = 5.5

Masukkan nilai b = 6.23

Bilangan terkecil antara 5 dan 6.23 adalah 5.5

Khusus untuk fungsi yang dirancang tanpa memberikan nilai keluaran (melainkan hanya menjalankan suatu tugas khusus) biasa didefinisikan dengan diawali kata kunci *void* (di depan nama fungsi). Sebagai contoh perhatikan program berikut ini.

```

* File program : void.c
Contoh fungsi tanpa nilai keluaran (pamakaian void) */
#include <stdio.h>
void info_program();      /* deklarasi fungsi */
main()
{
    info_program();      /* pemanggilan fungsi */
}

void info_program() /* definisi fungsi */
{
    puts("=====");
    puts("Progam dibuat oleh Moh. Izzuddin ");
    puts("Tanggal : 11 Agustus 2014 ");
    puts(" ");
}

```

```

    puts("Selamat menggunakannya.....");
    puts("=====");
}

```

Contoh eksekusi :

```

=====
Progam dibuat oleh Moh. Izzuddin
Tanggal : 11 Agustus 2014
Selamat menggunakannya.....
=====

```

6.5 Prototipe Fungsi

Prototipe fungsi digunakan untuk menjelaskan kepada kompiler mengenai :

- tipe keluaran fungsi
- jumlah parameter
- tipe dari masing-masing parameter.

Bagi kompiler, informasi dalam prototipe akan dipakai untuk memeriksa keabsahan (validitas) parameter dalam pemanggilan fungsi. Salah satu keuntungannya adalah, kompiler akan melakukan konversi seandainya antara tipe parameter dalam fungsi dan parameter saat pemanggilan fungsi tidak sama, atau akan menunjukkan kesalahan bila jumlah parameter dalam definisi dan saat pemanggilan berbeda.

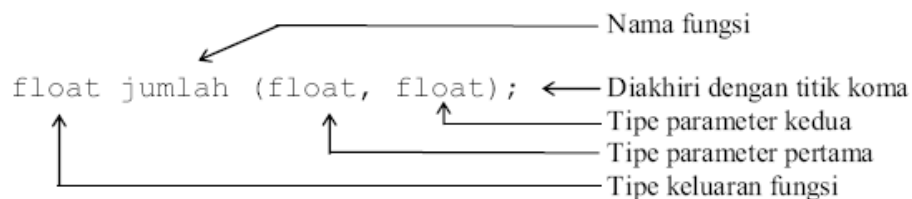
Contoh prototipe fungsi;

```
float jumlah (float x, float y);
```

atau

```
float jumlah (float, float);
```

Penjelasannya adalah sbb :



Gambar 6.4 Prototipe fungsi

Perhatikan contoh program berikut ini.

```
/* File program : jumlah.c
contoh pemakaian prototipe fungsi */ #include <stdio.h>
float jumlah(float, float); /* prototipe fungsi */
main()
{
    float a, b, c;
    printf("Masukkan nilai a : ");
    scanf("%f", &a);
    printf("Masukkan nilai b : "); scanf("%f", &b);
    c = jumlah(a, b);
    printf("\nHasil penjumlahan a + b = %g\n", c);
}

float jumlah(float x, float y) /* definisi fungsi
*/
{
    return(x + y);
}
```

Contoh eksekusi :

Masukkan nilai a : 4.5

Masukkan nilai b : 7.65

Hasil penjumlahan a + b = 12.15

Untuk fungsi yang tidak memiliki argumen (contoh program **void.c**), maka deklarasinya adalah

```
void info_program(void);
```



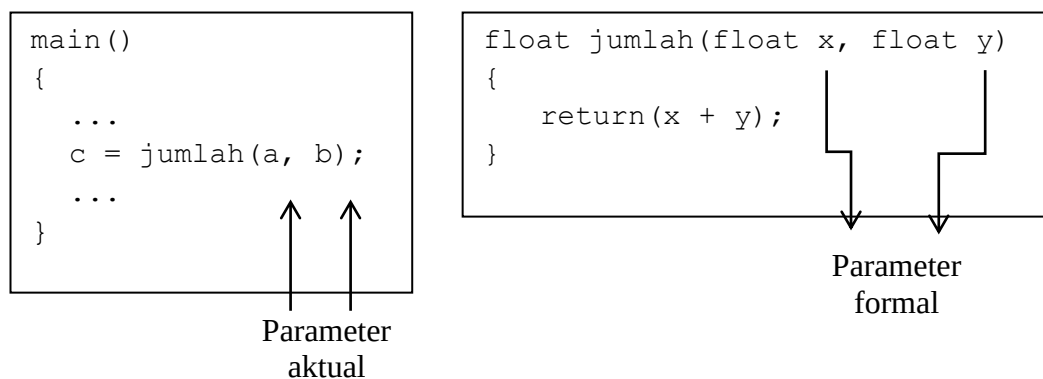
menyatakan bahwa info_program()
tidak memiliki parameter

Catatan :

- Untuk fungsi-fungsi pustaka, prototipe dari fungsi-fungsi berada di file-file judulnya (*header file*). Misalnya fungsi pustaka *printf()* dan *scanf()* prototipenya berada pada file dengan nama **stdio.h**
- Untuk fungsi pustaka pencantuman pada prototipe fungsi dapat dilakukan dengan menggunakan *preprocessor directive* **#include**.

6.6 Parameter Formal dan Parameter Aktual

Parameter formal adalah variabel yang ada pada daftar parameter dalam definisi fungsi. Pada contoh program di atas misalnya, maka dalam fungsi **jumlah()** variabel **x** dan **y** dinamakan sebagai parameter formal. Adapun parameter aktual adalah parameter (tidak selalu berupa variabel) yang dipakai dalam pemanggilan fungsi.



Gambar 6.5 Paramater formal dan parameter aktual

Pada pernyataan :

```
x = jumlah(a, b);  
y = jumlah(20.1, 45.6);
```

a dan **b** merupakan parameter aktual dari fungsi **jumlah()** dalam hal ini parameter berupa variabel. Demikian juga **20.1** dan **45.6** adalah parameter aktual, dalam hal ini berupa konstanta. Bahkan bisa juga parameter aktual berupa ungkapan yang melibatkan operator, misalnya :

```
printf("%g\n", jumlah(2+3, 3+6));
```

↳ ungkapan

6.7 Cara Melewatkan Parameter

Ada dua cara untuk melewati parameter kedalam fungsi. Cara yang dimaksud berupa;

- Pemanggilan dengan nilai (*call by value*),
- Pemanggilan dengan referensi (*call by reference*).

Pemanggilan dengan nilai merupakan cara yang dipakai untuk seluruh fungsi

buatan yang telah dibahas didepan. Pada pemanggilan dengan nilai, nilai dari parameter aktual akan disalin ke parameter formal. Dengan cara ini nilai parameter aktual tidak bisa dirubah sekalipun nilai parameter formal berubah. Untuk lebih jelasnya lihat pada fungsi **tukar()** pada contoh berikut ini.;

```
/* File program : tukar1.c
Untuk melihat pengaruh pemanggilan nilai pada fungsi
untuk penukaran dua data */
#include <stdio.h>
void tukar (int, int);
main()
{
    int a,b;
    a = 88; b = 77;
    printf("Nilai sebelum pemanggilan fungsi\n");
    printf("a = %d b = %d\n", a, b);
    tukar(a,b);
    printf("\nNilai setelah pemanggilan fungsi\n");
    printf("a = %d b = %d\n", a, b);
}

void tukar(int x, int y)
{
    int z;
    z = x;
    x = y;
    y = z;
    printf("\nNilai di akhir fungsi    tukar()\n");
    printf("x = %d y = %d\n", x, y);
}
```

Contoh eksekusi :

Nilai sebelum pemanggilan fungsi

a = 88 b = 77

Nilai di akhir fungsi tukar()

x = 77 y = 88

Nilai setelah pemanggilan fungsi

a = 88 b = 77

Tampak bahwa sekeduanya dari pemanggilan fungsi **tukar()**, variabel **a** dan **b** (yang dilewatkan ke fungsi **tukar()**) tidak berubah, walaupun pada fungsi **tukar()** telah terjadi penukaran antara parameter **x** dan **y**. Mengapa hal ini bisa terjadi? Sebab **x** hanyalah salinan dari **a** dan **y** adalah salinan dari **b** (Lihat gambar 6.6 di bawah ini). Pada saat pemanggilan fungsi, maka :

- **x** bernilai 88 (nilai **a**)
- **y** bernilai 77 (nilai **b**)

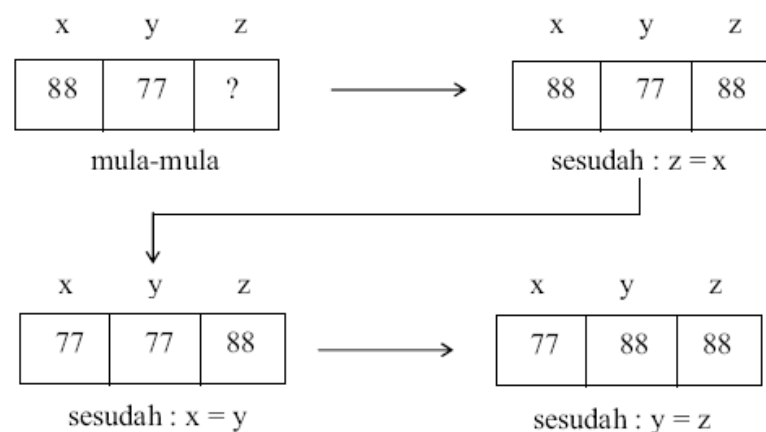
Sesudah pernyataan-pernyataan berikut dijalankan, maka :

`z = x;`

`x = y;`

`y = z;`

x akan bernilai 77 dan **y** bernilai 88.



Gambar 6.6 Proses penukaran nilai

Gambar 6.6 menjelaskan bahwa **a** dan **b** tidak berubah. Yang berubah hanyalah parameter **x** dan **y**.

Pemanggilan dengan referensi (*call by reference*) merupakan upaya untuk melewati alamat dari suatu variabel ke dalam fungsi. Cara ini dapat dipakai untuk mengubah isi suatu variabel di luar fungsi dengan pelaksanaan perubahan dilakukan di dalam fungsi. Sebagai contoh perhatikan program **tukar2.c** yang merupakan modifikasi dari **tukar1.c**. Perubahan yang pertama terletak dalam definisi fungsi, yang kini berupa;

```

void tukar(int *px, int *py)
{
    int z;
    z = *px; *px = *py;
    *py = z;
    printf("\nNilai di akhir fungsi tukar()\n");
    printf("x = %d y = %d\n", *px, *py);
}

```

Adapun perubahan dalam parameter aktualnya menjadi :

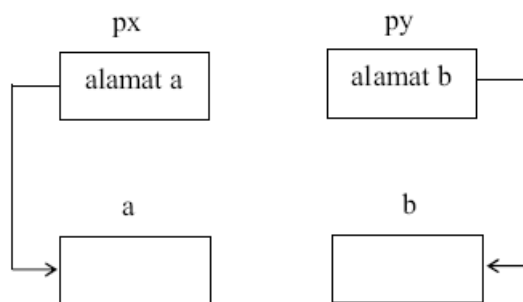
```
tukar(&a, &b); /* alamat a dan alamat b */
```

Dalam deklarasi parameter

```
int *px, int *py
```

menyatakan bahwa **px** dan **py** adalah suatu variabel pointer. Yang dimaksudkan sebagai variabel pointer adalah suatu variabel yang menunjuk ke variabel lain. Lebih jelasnya, variabel pointer berisi alamat dari variabel lain.

Adapun pada pemanggilan fungsi, **&a** dan **&b** masing-masing berarti "alamat a" dan "alamat b". Dengan pemanggilan seperti ini, hubungan antara variabel pointer **px** dan **py** dengan variabel **a** dan **b** adalah seperti ditunjukkan pada gambar 6.7. Dalam hal ini, **px** dikatakan menunjuk variabel **a** dan **py** menunjuk variabel **b**.



Gambar 6.7 Variabel pointer **px** menunjuk variabel **a** dan variabel pointer **py** menunjuk variabel **b**

```

/* File program : tukar2.c
Untuk melihat pengaruh pemanggilan nilai pada fungsi
untuk
penukaran dua data */
#include <stdio.h>
void tukar (int *px, int *py); /* prototype fungsi */

```



```

main()
{
    int a,b;
    a = 88;
    b = 77;
    printf("Nilai sebelum pemanggilan fungsi\n");
    printf("a = %d b = %d\n", a, b);
    tukar(&a,&b); /* alamat a dan alamat b */
    printf("\nNilai setelah pemanggilan fungsi\n");
    printf("a = %d b = %d\n", a, b);
}

void tukar(int *px, int *py)
{
    int z;
    z = *px;
    *px = *py;
    *py = z;
    printf("\nNilai di akhir fungsi tukar()\n");
    printf("x = %d y = %d\n", *px, *py);
}

```

Contoh eksekusi :

Nilai sebelum pemanggilan fungsi

a = 88 b = 77

Nilai di akhir fungsi tukar()

x = 77 y = 88

Nilai setelah pemanggilan fungsi

a = 77 b = 88

Setelah **px** menunjuk **a** dan **py** menunjuk **b**, proses penukaran isi **a** dan **b** dilakukan dengan cara sebagai berikut :

```

z = *px;          /* 1 */
*px = *py;        /* 2 */
*py = z;          /* 3 */

```

Pertama variabel **z** diisi dengan nilai yang ditunjuk oleh **px**. Kedua, yang ditunjuk oleh **px** diisi dengan yang ditunjuk oleh **py** (berarti **a** diisi dengan **b**). Ketiga, yang ditunjuk oleh **py** diberi nilai **z**. Dengan melalui tiga pernyataan di atas, nilai **a** dan **b** dapat diubah di dalam fungsi.

6.8 Penggolongan Variabel berdasarkan Kelas Penyimpanan

Suatu variabel, di samping dapat digolongkan berdasarkan jenis/tipe data juga dapat diklasifikasikan berdasarkan kelas penyimpanan (*storage class*). Penggolongan berdasarkan kelas penyimpanan berupa :

- variabel lokal
- variable eksternal
- variabel statis
- variabel register

6.8.1 Variabel Lokal

Variabel lokal adalah variabel yang dideklarasikan dalam fungsi, dengan sifat :

- Secara otomatis diciptakan ketika fungsi dipanggil dan akan sirna (lenyap) ketika eksekusi terhadap fungsi berakhir.
- Hanya dikenal oleh fungsi tempat variabel tersebut dideklarasikan
- Tidak ada inisialisasi secara otomatis (saat variabel diciptakan, nilainya tak menentu).

Variabel lokal disebut juga dengan variabel otomatis. Variabel yang termasuk dalam golongan ini bisa dideklarasikan dengan menambahkan kata kunci *auto* di depan tipe-data variabel. Kata kunci ini bersifat opsional, biasanya disertakan sebagai penjelas saja. Contoh variabel lokal ditunjukkan pada Gambar 6.8.

```
void fung_x(void)
{
    int x;
    .
    .
    .
}
```



x adalah variabel lokal bagi
fungsi fung_x()

Gambar 6.8 Variabel lokal

Pada **fung_x()**, deklarasi

```
int x;
```

dapat ditulis menjadi

```
auto int x;
```

Penerapan variabel lokal yaitu bila variabel hanya dipakai oleh suatu fungsi (tidak dimaksudkan untuk dipakai oleh fungsi yang lain). Pada contoh berikut, antara variabel **i** dalam fungsi **main()** dan **fung_1()** tidak ada kaitannya, sebab masing-masing merupakan variabel lokal.

```
/* File program : lokal.c */
#include <stdio.h>
void fung_1(void);
main()
{
    int i = 20;
    fung_1();
    printf("nilai i di dalam main() = %d\n", i);
}

void fung_1(void)
{
    int i = 11;
    printf("nilai i di dalam fung_1() = %d\n", i);
}
```

Contoh eksekusi :

nilai i di dalam fung_1() = 11

nilai i di dalam main() = 20

6.8.2 Variabel Eksternal

Variabel eksternal merupakan variabel yang dideklarasikan di luar fungsi, dengan sifat :

- Dapat diakses oleh semua fungsi
- Kalau tak diberi nilai, secara otomatis diinisialisasi dengan nilai sama dengan nol.

Contoh variabel eksternal ada pda program **ekstern1.c** yaitu berupa variabel **i**.

Pada pendeklarasian

```
int i = 273;
```

menyatakan bahwa **i** merupakan variabel eksternal dan diberi nilai awal sama dengan 273. Nilai dari variabel **i** selanjutnya dapat diubah oleh fungsi **tambah()** maupun **main()**. Setiap fungsi **tambah()** dipanggil maka nilai **i** akan bertambah satu.

```
/* File program : ekstern1.c
Contoh program dengan variabel eksternal */
#include <stdio.h>
int i = 273; /* variabel eksternal */
void tambah(void);
main()
{
    printf("Nilai awal i = %d\n", i);
    i += 7;
    printf("Nilai i kini = %d\n", i);
    tambah();
    printf("Nilai i kini = %d\n", i);
    tambah();
    printf("Nilai i kini = %d\n", i);
}
void tambah(void)
{
    i++;
}
```

Contoh eksekusi :

Nilai awal i = 273

Nilai i kini = 280

Nilai i kini = 281

Nilai i kini = 282

Pada contoh di atas, terlihat bahwa **i** hanya dideklarasikan di bagian atas program, dan tak dideklarasikan lagi dalam fungsi **main()** maupun **tambah()**. Oleh karena **i** merupakan variabel eksternal maka dapat digunakan oleh kedua fungsi tsb. Namun ada satu hal yang perlu diketahui, variabel eksternal haruslah dideklarasikan sebelum definisi fungsi yang akan mempergunakannya.

Untuk memperjelas bahwa suatu variabel dalam fungsi merupakan variabel eksternal, di dalam fungsi yang menggunakannya dapat mendeklarasikan

variabel itu kembali dengan menambahkan kata kunci `extern` di depan tipe data variabel. Sebagai contoh, program **ekstern1.c** ditulis kembali menjadi seperti pada **ekstern2.c**.

```
/* File program : ekstern2.c
Contoh program yang menggunakan variabel eksternal dan
memakai kata kunci extern */
#include <stdio.h>
int i = 273; /* variabel eksternal */
void tambah(void);
main()
{
    extern int i; /* variabel eksternal */
    printf("Nilai awal i = %d\n", i);
    i += 7;
    printf("Nilai i kini = %d\n", i);
    tambah();
    printf("Nilai i kini = %d\n", i);
    tambah();
    printf("Nilai i kini = %d\n", i);
}
void tambah(void)
{
    extern int i; /* variabel eksternal */
    i++;
}
```

Contoh eksekusi :

Nilai awal i = 273

Nilai i kini = 280

Nilai i kini = 281

Nilai i kini = 282

Kalau dalam suatu program terdapat suatu variabel eksternal, suatu fungsi bisa saja menggunakan nama variabel yang sama dengan variabel eksternal, namun diperlakukan sebagai variabel lokal. Untuk lebih jelasnya perhatikan contoh program berikut ini.

```

/* File program : ekstern3.c
Contoh program yang menggunakan variabel eksternal dan
variabel lokal dengan nama yang sama */
#include <stdio.h>
int i = 273; /* variabel eksternal */
void tambah(void);
main()
{
    extern int i; /* variabel eksternal */
    printf("Nilai awal i = %d\n", i);
    i += 7;
    printf("Nilai i kini = %d\n", i);
    tambah();
    printf("Nilai i kini = %d\n", i);
    tambah();
    printf("Nilai i kini = %d\n", i);
}
void tambah(void)
{
    int i; /* variabel lokal */
    i++;
}

```

Contoh eksekusi :

Nilai awal i = 273

Nilai i kini = 280

Nilai i kini = 280

Nilai i kini = 280

Pada program di atas, bagi fungsi **main()** **i** adalah variabel eksternal. Namun bagi fungsi **tambah()**, **i** merupakan variabel lokal, sebab pada fungsi ini **i** dideklarasikan tanpa kata kunci *extern*. Hal ini terlihat jelas dengan mengamati hasil eksekusi program. Pernyataan:

```
i++;
```

Pada fungsi **tambah()** tidak mempengaruhi nilai **i** yang ditampilkan pada fungsi **main()** (bandingkan dengan hasil eksekusi pada **ekstern2.c**).

6.8.3 Variabel Statis

Variabel statis dapat berupa variabel internal (didefinisikan di dalam fungsi) maupun variabel eksternal. Sifat variabel ini :

- Kalau variabel statis bersifat internal, maka variabel hanya dikenal oleh fungsi tempat variabel dideklarasikan.
- Kalau variabel statis bersifat eksternal, maka variabel dapat dipergunakan oleh semua fungsi yang terletak pada file yang sama, tempat variabel statis dideklarasikan.
- Berbeda dengan variabel lokal, variabel statis tidak akan hilang sekeluarnya dari fungsi (nilai pada variabel akan tetap diingat).
- Inisialisasi akan dilakukan hanya sekali, yaitu saat fungsi dipanggil yang pertama kali. Kalau tak ada inisialisasi oleh pemrogram secara otomatis akan diberi nilai awal nol.

Variabel statis diperoleh dengan menambahkan kata kunci static di depan tipe data variabel. Sebagai contoh perhatikan program di bawah ini.

```
/* File program : statis.c
Contoh variabel statis */
#include <stdio.h>
void fung_y(void);
main()
{
    int y = 20;
    fung_y();
    fung_y();
    printf("Nilai y dalam main() = %d\n", y);
}
void fung_y(void)
{
    static int y;
    y++;
    printf("Nilai y dalam fung_y() = %d\n", y);
}
```

Contoh eksekusi :

Nilai y dalam fung_y() = 1

Nilai y dalam `fung_y()` = 2

Nilai y dalam `main()` = 20

6.8.4 Variabel Register

Variabel register adalah variabel yang nilainya disimpan dalam register dan bukan dalam memori RAM. Variabel yang seperti ini hanya bisa diterapkan pada variabel yang lokal atau parameter formal, yang bertipe *char* atau *int*. Variabel register biasa diterapkan pada variabel yang digunakan sebagai pengendali loop. Tujuannya untuk mempercepat proses dalam *loop*. Sebab variabel yang dioperasikan pada register memiliki kecepatan yang jauh lebih tinggi daripada variabel yang diletakkan pada RAM. Contoh pemakaiannya bisa dilihat pada program berikut ini.

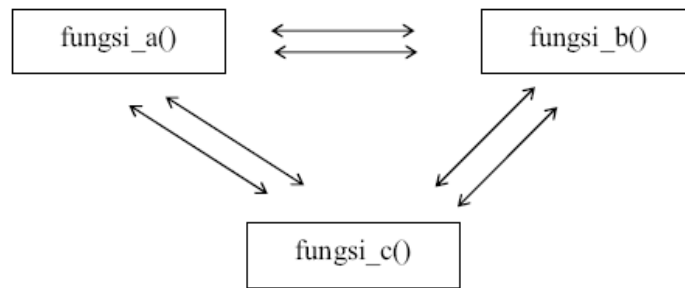
```
/* File program : var_reg.c
Contoh variabel register */
#include <stdio.h>
main()
{
    register int i; /* variabel register */
    int jumlah = 0;
    for(i = 1; i <= 100; i++)
        jumlah = jumlah + i;
    printf("1 + 2 + 3 + ... + 100 = %d\n", jumlah);
}
```

Contoh eksekusi :

1 + 2 + 3 + ... + 100 = 5050

6.9 Menciptakan Sejumlah Fungsi

Pada C, semua fungsi bersifat sederajat. Suatu fungsi tidak dapat didefinisikan di dalam fungsi yang lain. Akan tetapi suatu fungsi diperbolehkan memanggil fungsi yang lain, dan tidak tergantung kepada peletakan definisi fungsi pada program. Komunikasi antara fungsi dalam C ditunjukkan dalam gambar 6.9. Gambar tersebut menjelaskan kalau suatu fungsi katakanlah **fungsi_a()** memanggil **fungsi_b()**, maka bisa saja **fungsi_b()** memanggil **fungsi_a()**. Contoh program yang melibatkan fungsi yang memanggil fungsi yang lain ada pada program **kom_fung.c**, yaitu **fungsi_1()** dipanggil dalam **main()**, sedangkan **fungsi_2()** dipanggil oleh **fungsi_1()**.



Gambar 6.9 Komunikasi antar fungsi dalam C

```

/* File program : kom_fung.c
contoh fungsi yang memanggil fungsi yang lain */
#include <stdio.h>
void fungsi_1(void);
void fungsi_2(void);
main()
{
    fungsi_1();
}
void fungsi_1()
{
    puts("fungsi 1 dijalankan");
    fungsi_2();
}
void fungsi_2()
{
    puts("fungsi 2 dijalankan");
}
  
```

Contoh eksekusi :

fungsi 1 dijalankan

fungsi 2 dijalankan

6.10 Rekursi

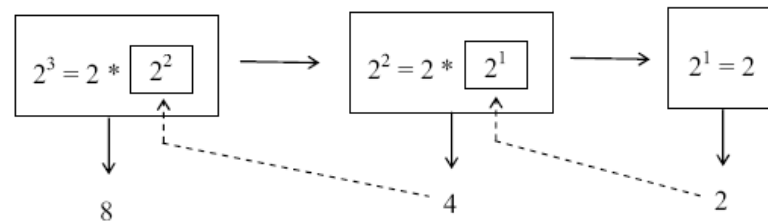
Fungsi dalam C dapat dipakai secara rekursi, dalam artian suatu fungsi dapat memanggil dirinya sendiri. Sebagai contoh penerapan fungsi rekursi yaitu untuk menghitung nilai

$$x^n$$

dengan n berupa bilangan bulat positif. Solusi dari persoalan ini dapat berupa :

- Jika $n = 1$, maka $x^n = x$
- Selain itu maka $x^n = x * x^{n-1}$

Misalnya $x = 2$ dan $n = 3$, proses pemecahannya seperti diuraikan pada gambar 6.10.



Gambar 6.10 Pemecahan secara rekursi

Penuangan dalam bentuk program

```
/* File program : faktor.c
Contoh penerapan rekursi untuk memperoleh nilai
factorial */
#include <stdio.h>
int faktorial(int);
main()
{
    int x;
    puts("MENCARI FAKTORIAL DARI X!");
    printf("Masukkan nilai x (bulat positif) : ");
    scanf("%d", &x);
    printf("Faktorial dari %d = %d\n", x,
    faktorial(x));
}
int faktorial(int m)
{
    if(m == 1)
        return(1);
    else
        return(m * faktorial(m-1));
}
```

Contoh eksekusi :

MENCARI FAKTORIAL DARI X!

Masukkan nilai x (bulat positif) : 4

Faktorial dari $4 = 24$

Rekursi jarang dipakai, di antaranya disebabkan :

- Biasanya rekursi akan menjadikan fungsi sulit dimengerti. Hanya cocok untuk persoalan tertentu saja (misalnya pada *binary tree* atau pohon biner). Untuk fungsi rekursi pada program faktor.c di atas misalnya, akan lebih mudah dipahami kalau ditulis menjadi :

```
int faktorial(int m)
{
    int i, fak;
    fak = 1;
    for(i = 1; i <= m; i++)
        fak = fak * i;
    return(fak);
}
```

- Memerlukan *stack* dengan ukuran yang lebih besar. Sebab setiap kali fungsi dipanggil, variabel lokal dan parameter formal akan ditempatkan ke *stack* dan adakalanya akan menyebabkan *stack* tak cukup lagi (*stack overflow*).

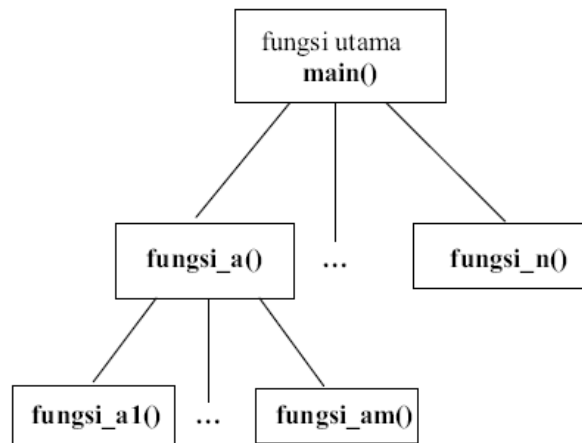
6.11 Pengenalan Konsep Pemrograman Terstruktur

Fungsi sangat bermanfaat untuk membuat program yang terstruktur. Suatu program yang terstruktur dikembangkan dengan menggunakan “*top-down design*” (rancang atas bawah). Pada C suatu program disusun dari sejumlah fungsi dengan tugas tertentu. Selanjutnya masing masing fungsi dipecah-pecah lagi menjadi fungsi yang lebih kecil. Pembuatan program dengan cara ini akan memudahkan dalam pencarian kesalahan ataupun dalam hal pengembangan dan tentu saja mudah dipahami/ dipelajari.

Dalam bentuk diagram, model suatu program C yang terstruktur adalah seperti yang tertera pada bagan berikut ini. Namun sekali lagi perlu diketahui, bahwa pada C semua fungsi sebenarnya berkedudukan sederajat.

Fungsi **main()** terdiri dari **fungsi_a()** sampai dengan **fungsi_n()**, menegaskan bahwa dalam program fungsi **main()** akan memanggil **fungsi_a()**

sampai dengan **fungsi_n()**. Adapun fungsi-fungsi yang dipanggil oleh fungsi **main()** juga bisa memanggil fungsi-fungsi yang lain.



Gambar 6.11 Model terstruktur Program C

Kesimpulan

- Fungsi digunakan untuk memecah program yang besar menjadi program-program kecil sesuai dengan fungsi masing-masing.
- Fungsi bisa memberikan nilai balik dan bisa tanpa memberikan nilai balik kepada fungsi yang memanggilnya.
- Fungsi yang memberikan nilai balik harus memiliki tipe dan ditulis didepan nama fungsi.
- Bila fungsi tidak memberikan nilai balik maka fungsi tersebut bertipe “void” dan ditulis didepan nama fungsi.

Latihan :

Buatlah potongan program untuk soal-soal di bawah ini

1. Buatlah sebuah fungsi yang berfungsi untuk menampilkan sebuah string (di layar) = “*Pilihan Menu*” (misalkan nama fungsinya = **menu**). Fungsi tersebut tidak memiliki nilai kembalian (*return value*) dan juga tidak menerima parameter masukan apapun.
2. Tulislah prototipe fungsi untuk fungsi pada soal nomor 1 di atas.
3. Buatlah sebuah fungsi (misalkan nama fungsinya = **cetak**) yang berfungsi untuk menampilkan sebuah string (di layar). Fungsi tersebut tidak memiliki

nilai kembalian (*return value*), tetapi menerima parameter masukan berupa string yang akan dicetak (catatan : string merupakan array karakter).

4. Tulislah prototipe fungsi untuk fungsi pada soal nomor 3 di atas.
5. Buatlah sebuah fungsi (misalkan nama fungsinya = **total**) yang berfungsi untuk menjumlah total nilai dari array integer yang dikirim sebagai parameter masukan fungsi tsb. Fungsi tersebut memberikan nilai kembalian (*return value*) bertipe integer yang berisi total hasil perhitungannya. Dalam hal ini fungsi tsb memiliki 2 parameter masukan berupa array integer dan sebuah variabel integer yang menunjukkan jumlah elemen dari array tsb.
6. Tulislah prototipe fungsi untuk fungsi pada soal nomor 5 di atas.

BAB VII

ARRAY

7.1 Tujuan

- 1 Menjelaskan tentang array berdimensi satu
- 2 Menjelaskan tentang array berdimensi dua
- 3 Menjelaskan tentang array berdimensi banyak
- 4 Menjelaskan tentang inisialisasi array tak berukuran.
- 5 Menjelaskan array sebagai parameter fungsi

7.2 Defenisi Array

Dalam beberapa literatur, array sering disebut (diterjemahkan) sebagai larik. Array adalah kumpulan dari nilai-nilai data bertipe sama dalam urutan tertentu yang menggunakan sebuah nama yang sama. Nilai-nilai data di suatu array disebut dengan elemen-elemen array. Letak urutan dari elemen-elemen array ditunjukkan oleh suatu *subscript* atau indeks.

Array bisa berupa array berdimensi satu, dua, tiga atau lebih. Array berdimensi satu (*one-dimensional array*) mewakili bentuk suatu vektor. Array berdimensi dua (*two-dimensional array*) mewakili bentuk dari suatu matriks atau table. Array berdimensi tiga (*three-dimensional array*) mewakili bentuk suatu ruang.

7.3 Array Berdimensi Satu

7.3.1 Mendeklarasikan Array Berdimensi Satu

Suatu array berdimensi satu dideklarasikan dalam bentuk umum berupa :

`tipe_data nama_var[ukuran];`

dengan :

- `tipe_data` : untuk menyatakan tipe dari elemen array, misalnya `int`, `char`, `float`.
- `nama_var` : nama variabel array
- `ukuran` : untuk menyatakan jumlah maksimal elemen array.

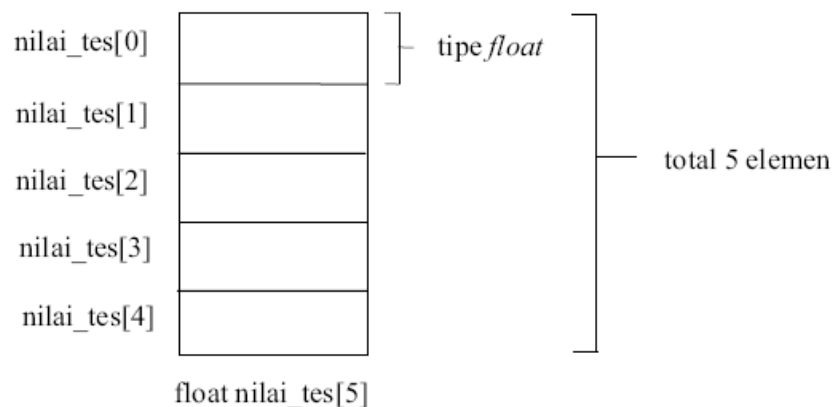
Contoh pendeklarasian array :

```
float nilai_tes[5];
```

menyatakan bahwa array **nilai_tes** mengandung 5 elemen bertipe *float*.

7.3.2 Mengakses Elemen Array Berdimensi Satu

Pada C, data array akan disimpan dalam memori yang berurutan. Elemen pertama mempunyai indeks bernilai 0. Jika variabel **nilai_tes** dideklarasikan sebagai array dengan 5 elemen, maka elemen pertama memiliki indeks sama dengan 0, dan elemen terakhir memiliki indeks 4. Gambar 7.1 di bawah ini menjelaskan urutan komponen dalam array.



Gambar 7.1 Array berdimensi satu

Bentuk umum pengaksesan array adalah sbb :

```
nama_var[indeks]
```

sehingga, untuk array **nilai_tes**, maka :

`nilai_tes[0]` → elemen pertama dari

nilai_tes `nilai_tes[4]` → elemen ke-

5 dari **nilai_tes**

Contoh :

```
nilai_tes[0] = 70;           /* contoh 1 */  
scanf("%f", &nilai_tes[2]); /* contoh 2 */
```

Contoh pertama merupakan pemberian nilai 70 ke **nilai_tes[0]**. Sedangkan contoh 2 merupakan perintah untuk membaca data bilangan dari keyboard dan diberikan ke **nilai_tes[2]**. Pada contoh 2 ini

```
&nilai_tes[2]
```

berarti “alamat dari **nilai_tes[2]**”. Perlu diingat bahwa *scanf()* memerlukan argumen berupa alamat dari variabel yang digunakan untuk menyimpan nilai masukan. Selengkapnya perhatikan contoh program di bawah ini

```
/* File program : nilai_tes.c
Pemakaian array utk menyimpan sejumlah nilai tes */
#include <stdio.h>
#define MAKS 5
main()
{
    int i;
    float total = 0, rata;
    float nilai_tes[MAKS]; /* deklarasi array */
    for(i=0; i < MAKS; i++) /* pemasukan data
    nilai_tes */
    {
        printf("Nilai tes ke-%d : ", i+1);
        scanf("%f", &nilai_tes[i]);
        /* menghitung jumlah seluruh nilai */
        total = total + nilai_tes[i];
    }
    rata = total / MAKS; /* hitung nilai rata-rata */
    /* cetak nilai rata-rata */
    printf("\nNilai rata-rata = %g\n", rata);
}
```

Contoh eksekusi :

Nilai tes ke-1 : 56.5

Nilai tes ke-2 : 67.75

Nilai tes ke-3 : 80

Nilai tes ke-4 : 77

Nilai tes ke-5 : 78.5

Nilai rata-rata = 71.95

7.3.3 Inisialisasi Array Berdimensi Satu

Sebuah array dapat diinisialisasi sekaligus pada saat dideklarasikan. Untuk mendeklarasikan array, nilai-nilai yang diinisialisasikan dituliskan di antara kurung kurawal ({}) yang dipisahkan dengan koma.

```
/* File program : jhari.c */
#include <stdio.h>
main()
{
    int bulan, tahun, jhari;
    int jum_hari[12] =
    {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    puts("MEMPEROLEH JUMLAH HARI");
    puts("PADA SUATU BULAN DAN SUATU TAHUN");
    puts("-----");
    printf("Masukkan bulan (1..12) : ");
    scanf("%d", &bulan);
    printf("Masukkan tahunnya : ");
    scanf("%d", &tahun);
    if(bulan == 2)
        if(tahun % 4 == 0)
            jhari = 29;
        else
            jhari = 28;
    else
        jhari = jum_hari[bulan-1];
    printf("\nJumlah hari dalam bulan %d tahun %d
    adalah %d
    hari\n", bulan, tahun, jhari);
}
```

Contoh eksekusi :

```
MEMPEROLEH JUMLAH HARI
PADA SUATU BULAN DAN SUATU TAHUN
-----
Masukkan bulan (1..12) : 2
Masukkan tahunnya : 1988
Jumlah hari dalam bulan 2 tahun 1988 adalah 29 hari
```

Pada program **jhari.c**

```
int jum_hari[12] =  
{31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

merupakan instruksi untuk mendeklarasikan array **jum_hari** yang memiliki 12 elemen yang bersifat statis dan sekaligus melakukan inisialisasi terhadap masing-masing elemen array.

```
/* File program : inisial.c */  
#include <stdio.h>  
main()  
{  
    int i;  
    int values[] = {1,2,3,4,5,6,7,8,9};  
    char word[] = {'H','e','l','l','o'};  
    for(i = 0; i < 9; ++i )  
        printf("values[%d] is %d\n", i, values[i]);  
    printf("\n");  
    for(i = 0; i < 6; ++i )  
        printf("word[%d] is %c\n", i, word[i]);  
}
```

Contoh eksekusi :

values[0] is 1

values[1] is 2

values[2] is 3

values[3] is 4

values[4] is 5

values[5] is 6

values[6] is 7

values[7] is 8

values[8] is 9

word[0] is H

word[1] is e

word[2] is l

word[3] is l

word[4] is o

Perhatikan, pada contoh **inisial.c**, pendeklarasian nama variabel array tidak disertai ukuran yang mengindikasikan besarnya array. Dalam kondisi seperti ini, C akan menginisialisasi ukuran array tersebut sejumlah elemen yang diberikan di dalam kurung kurawal pada saat proses inisialisasi. Sehingga array **values** terdiri atas 9 elemen dan array **word** memiliki 5 elemen.

7.3.4 Beberapa Variasi dalam Mendeklarasikan Array

Ada beberapa variasi cara mendeklarasikan sebuah array (dalam hal ini yang berdimensi satu), di antaranya adalah sebagai berikut :

- `int numbers[10];`
- `int numbers[10] = { 34, 27, 16 };`
- `int numbers[] = { 2, -3, 45, 79, -14, 5, 9, 28, -1, 0 };`
- `char text[] = "Welcome to New Zealand.";`
- `float radix[12] = { 134.362, 1913.248 };`
- `double radians[1000];`

7.4 Array Berdimensi Dua

Data seperti yang disajikan pada Tabel 7.1, dapat disimpan pada sebuah array berdimensi dua. Dimensi pertama dari array digunakan untuk menyatakan kode program kursus dan dimensi kedua untuk menyatakan tahun kursus.

Tabel 7.1. Data Kelulusan Siswa Pada Sebuah Kursus Komputer

Tahun Program	1998	1999	2000
1. (INTRO)	80	540	1032
2. (BASIC)	15	83	301
3. (PASCAL)	8	12	15
4. (C)	10	129	257

7.4.1 Mendeklarasikan Array Berdimensi Dua

Pendeklarasian yang diperlukan untuk menyimpan data kelulusan siswa pada Tabel 6.1 adalah:

```
int data_lulus[4][3];
```

Nilai 3 untuk menyatakan banyaknya tahun dan 4 menyatakan banyaknya program kursus. Gambar 7.2 memberikan ilustrasi untuk memudahkan pemahaman tentang array berdimensi dua.

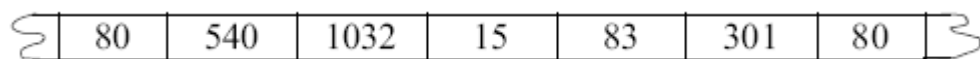
	0	1	2	← indeks kedua (tahun)
0	80	540	1032	
1	15	83	301	
2	8	12	15	
3	10	129	257	

↑ indeks pertama
(program kursus)

`int data_lulus[4][3];`

Gambar 7.2 Array berdimensi dua

Sama halnya pada array berdimensi satu, data array akan ditempatkan pada memori yang berurutan. Perhatikan Gambar 6.3.



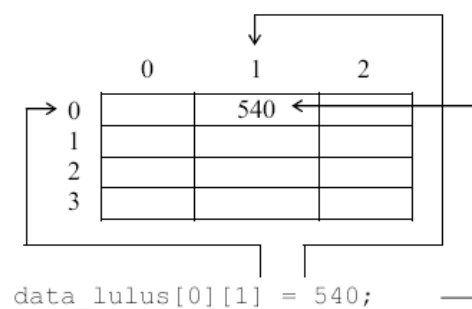
Gambar 6.3 Model penyimpanan array dimensi dua pada memori

7.4.2 Mengakses Elemen Array Berdimensi Dua

Array seperti **data_lulus** dapat diakses dalam bentuk

`data_lulus[indeks pertama, indeks kedua]`

Contoh :



Gambar 7.4. Pemberian nilai ke array berdimensi dua

(1) `data_lulus[0][1] = 540;`

merupakan instruksi untuk memberikan nilai 540 ke array **data_lulus** untuk indeks pertama = 0 dan indeks kedua bernilai 1.

```
(2) printf("%d", data_lulus[2][0]);
```

merupakan perintah untuk menampilkan elemen yang memiliki indeks pertama = 2 dan indeks kedua = 0.

Perhatikan contoh program berikut ini.

```
/* File program : lulus.c
Contoh pemakaian array berdimensi dua */
#include <stdio.h>
main( )
{
    int tahun, kode_program;
    int data_lulus[4][3] ;

    /* Memberikan data ke array */
    data_lulus[0][0] = 80;
    data_lulus[0][1] = 540;
    data_lulus[0][2] = 1032;
    data_lulus[1][0] = 15;
    data_lulus[1][1] = 83;
    data_lulus[1][2] = 301;
    data_lulus[2][0] = 8;
    data_lulus[2][1] = 12;
    data_lulus[2][2] = 15;
    data_lulus[3][0] = 10;
    data_lulus[3][1] = 129;
    data_lulus[3][2] = 257;
    /* proses utk memperoleh informasi jml siswa yg
    lulus */
    printf("Masukkan tahun dr data yg ingin anda
    ketahui ");
    printf("(1998..2000) : ");
    scanf("%d", &tahun);
    printf("Masukkan kode program kursus yang ingin
    anda ketahui");
    printf("(1 = INTRO, 2 = BASIC, 3 = PASCAL, 4 = C)
    : ");
    scanf("%d", &kode_program);
    printf("\nTotal kelulusan program tsb = %d\n",
    data_lulus[kode_program - 1][tahun - 1998] );
}
```

Contoh eksekusi :

Masukkan tahun dr data yg ingin anda ketahui (1998...2000) : 2000

Masukkan kode program kursus dari data yang ingin anda ketahui (1 = INTRO, 2 = BASIC, 3 = PASCAL, 4 = C) : 3

Total kelulusan program tsb = 15

7.4.3 Inisialisasi Array Berdimensi Dua

Gambar berikut memberikan penjelasan tentang inisialisasi yang dilakukan terhadap array berdimensi dua :

0	1	1	1	1	1	0	0
0	1	0	0	0	1	0	0
0	1	0	0	0	1	0	0
1	1	1	1	1	1	1	0
1	1	0	0	0	0	1	0
1	1	0	0	0	0	1	0
1	1	0	0	0	0	1	0
0	0	0	0	0	0	0	0

Gambar 7.5 Representasi inisialisasi array berdimensi dua

Untuk itu, deklarasi dan inisialisasi yang dilakukan berupa :

```
int huruf_A[8][8] = {  
    { 0, 1, 1, 1, 1, 1, 0, 0 } ,  
    { 0, 1, 0, 0, 0, 1, 0, 0 } ,  
    { 0, 1, 0, 0, 0, 1, 0, 0 } ,  
    { 1, 1, 1, 1, 1, 1, 1, 0 } ,  
    { 1, 1, 0, 0, 0, 0, 1, 0 } ,  
    { 1, 1, 0, 0, 0, 0, 1, 0 } ,  
    { 1, 1, 0, 0, 0, 0, 1, 0 } ,  
    { 0, 0, 0, 0, 0, 0, 0, 0 }  
};
```

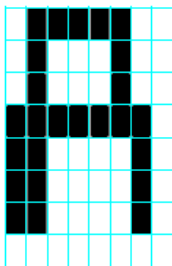
atau bisa juga ditulis sebagai berikut :

```
int huruf_A[8][8] =  
    { 0, 1, 1, 1, 1, 1, 0, 0,  
      0, 1, 0, 0, 0, 1, 0, 0,  
      0, 1, 0, 0, 0, 1, 0, 0,  
      1, 1, 1, 1, 1, 1, 1, 0,  
      1, 1, 0, 0, 0, 0, 1, 0,  
      1, 1, 0, 0, 0, 0, 1, 0,  
      1, 1, 0, 0, 0, 0, 1, 0,  
      0, 0, 0, 0, 0, 0, 0, 0  
    };
```

Contoh program berikut memanfaatkan data yang ada pada array **huruf_A** untuk membentuk karakter A dengan ukuran besar. Setiap nilai satu pada array akan diganti dengan karakter ber-ASCII 219 (DBh) dan nilai 0 akan diganti dengan karakter spasi.

```
/* File program : hurufA.c
Contoh inisialisasi array dimensi dua */
#include <stdio.h>
main()
{
    int i,j;
    int huruf_A[8][8] = {
        { 0, 1, 1, 1, 1, 1, 0, 0 } ,
        { 0, 1, 0, 0, 0, 1, 0, 0 } ,
        { 0, 1, 0, 0, 0, 1, 0, 0 } ,
        { 1, 1, 1, 1, 1, 1, 1, 0 } ,
        { 1, 1, 0, 0, 0, 0, 1, 0 } ,
        { 1, 1, 0, 0, 0, 0, 1, 0 } ,
        { 1, 1, 0, 0, 0, 0, 1, 0 } ,
        { 0, 0, 0, 0, 0, 0, 0, 0 }
    };
    for(i = 0; i < 8; i++)
    {
        for(j = 0; j < 8; j++)
            if(huruf_A[i][j] !=0 )
                putchar('\xDB\n');
            else
                putchar (' '); /* spasi */
            putchar('\n');
    }
}
```

Contoh eksekusi :



7.5 Array Berdimensi Banyak.

C memungkinkan untuk membuat array yang dimensinya lebih dari dua.

Bentuk umum pendeklarasian array berdimensi banyak :

```
tipe nama_var[ukuran 1][ukuran2]...[ukuranN];
```

sebagai contoh :

```
int data_huruf[2][8][8];
```

merupakan pendeklarasian array **data_huruf** sebagai array berdimensi tiga.

Sama halnya dengan array berdimensi satu atau dua, array berdimensi banyak juga bisa diinisialisasi. Contoh inisialisasi array berdimensi tiga :

Program berikut merupakan pengembangan dari Program 8 – 4, digunakan untuk menampilkan dua buah huruf dengan ukuran besar.

```
int data_huruf [2][8][8] =
{ { { 0, 1, 1, 1, 1, 1, 0, 0 } ,
    { 0, 1, 0, 0, 0, 1, 0, 0 } ,
    { 0, 1, 0, 0, 0, 1, 0, 0 } ,
    { 1, 1, 1, 1, 1, 1, 1, 0 } ,
    { 1, 1, 0, 0, 0, 0, 1, 0 } ,
    { 1, 1, 0, 0, 0, 0, 1, 0 } ,
    { 1, 1, 0, 0, 0, 0, 1, 0 } ,
    { 0, 0, 0, 0, 0, 0, 0, 0 }
  },
  { {1, 1, 1, 1, 1, 1, 0, 0 } ,
    {1, 0, 0, 0, 0, 1, 0, 0 } ,
    {1, 0, 0, 0, 0, 1, 0, 0 } ,
    {1, 1, 1, 1, 1, 1, 1, 0 } ,
    {1, 1, 0, 0, 0, 0, 1, 0 } ,
    {1, 1, 0, 0, 0, 0, 1, 0 } ,
    {1, 1, 1, 1, 1, 1, 1, 0 } ,
    {0, 0, 0, 0, 0, 0, 0, 0 }
  }
};
```

atau bisa juga ditulis menjadi

```
int data_huruf [2][8][8] =
{ 0, 1, 1, 1, 1, 1, 0, 0,
  0, 1, 0, 0, 0, 1, 0, 0,
  0, 1, 0, 0, 0, 1, 0, 0,
  1, 1, 1, 1, 1, 1, 1, 0,
```



```

1, 1, 0, 0, 0, 0, 1, 0,
1, 1, 0, 0, 0, 0, 1, 0,
1, 1, 0, 0, 0, 0, 1, 0,
0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 1, 1, 0, 0,
1, 0, 0, 0, 0, 1, 0, 0,
1, 0, 0, 0, 0, 1, 0, 0,
1, 1, 1, 1, 1, 1, 1, 0,
1, 1, 0, 0, 0, 0, 1, 0,
1, 1, 0, 0, 0, 0, 1, 0,
1, 1, 1, 1, 1, 1, 1, 0,
0, 0, 0, 0, 0, 0, 0, 0
};

```

Program berikut merupakan pengembangan dari Program 8.4, digunakan untuk menampilkan dua buah huruf dengan ukuran besar.

```

/* File program: data_huruf.c
Contoh inisialisasi array dimensi tiga */
# include <stdio.h>
main()
{
    int i, j, k;
    int data_huruf[2][8][8] = {
        {{ 0, 1, 1, 1, 1, 1, 0, 0 },
         { 0, 1, 0, 0, 0, 1, 0, 0 },
         { 0, 1, 0, 0, 0, 1, 0, 0 },
         { 1, 1, 1, 1, 1, 1, 1, 0 },
         { 1, 1, 0, 0, 0, 0, 1, 0 },
         { 1, 1, 0, 0, 0, 0, 1, 0 },
         { 1, 1, 0, 0, 0, 0, 1, 0 },
         { 0, 0, 0, 0, 0, 0, 0, 0 }
        },
        {{1, 1, 1, 1, 1, 1, 0, 0 },
         {1, 0, 0, 0, 0, 1, 0, 0 },
         {1, 0, 0, 0, 0, 1, 0, 0 },
         {1, 1, 1, 1, 1, 1, 1, 0 },
         {1, 1, 0, 0, 0, 0, 1, 0 },
         {1, 1, 0, 0, 0, 0, 1, 0 },
         {1, 1, 1, 1, 1, 1, 1, 0 },
         {0, 0, 0, 0, 0, 0, 0, 0 }
        }
    }
}

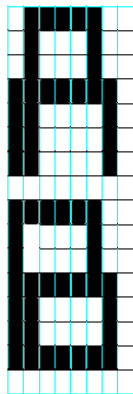
```

```

    };
    for(i = 0; i < 2; i++) /* Tampilkan huruf */
    {
        for(j = 0; j < 8; j++)
        {
            for(k = 0; k < 8; k++)
                if(data_huruf[i][j][k] != 0)
                    putchar('\xDB');
            else
                putchar(' '); /* spasi */
            printf("\n"); /* pindah baris */
        }
        printf("\n"); /* pindah baris */
    }
}

```

Contoh Eksekusi :



7.6 Inisialisasi Array Tak Berukuran

Inisialisasi array yang tak berukuran dapat dilakukan untuk array berdimensi satu atau lebih. Untuk array berdimensi lebih dari satu, dimensi terkirilah yang boleh tak berukuran. Dengan cara ini tabel dalam array dapat diperluas atau dikurangi tanpa mengubah ukuran array. Sebagai contoh :

```

int skala[] ← tak berukuran
{ 1, 2, 4, 6, 8 };

```

merupakan pendeklarasian array berdimensi satu yang tak berukuran. Secara otomatis

skala[0] bernilai 1

skala[1] bernilai 2

skala[2] bernilai 4

skala[3] bernilai 6

Contoh lain :

```
char konversi[][2] =
    { 'A', 'T',
      'E', 'M',
      'I', 'V',
      'O', 'S',
      'U', 'J',
    };
```

Pada contoh ini,

konversi[0][0]

bernilai 'A'

konversi[0][1]

bernilai 'T'

konversi[1][0]

bernilai 'E'

konversi[1][1]

bernilai 'M'

.....

Contoh berikut akan menyandikan suatu kalimat yang dimasukkan melalui keyboard dengan menggunakan data yang ada pada tabel **konversi**. Misal, huruf A akan diganti menjadi T, huruf T akan diganti menjadi A.

```
/* File program : sandi.c
contoh inisialisasi array tak berukuran */
#include <stdio.h>
#include <stdlib.h>
#define JUM_KOLOM 2
#define MAX_KAR 256
main()
{
    char konversi[][JUM_KOLOM] = {
        'A', 'T',
        'a', 't',
```

```

        'E', 'M',
        'e', 'm',
        'I', 'V',
        'i', 'v',
        'O', 'S',
        'o', 's',
        'U', 'J',
        'u', 'j'
    };
    char kalimat[MAX_KAR], karakter;
    int i = 0, j, jum_kar, jum_penyandi;
    printf("Masukkan sebuah kalimat dan akhiri dengan ENTER\n");
    printf("Kemudian      kalimat      tsb      akan      saya sandikan\n");
    printf("Kalimat: ");

    /* Memasukkan data karakter ke array kalimat */
    while((kalimat[i] = getchar()) != '\n')
        i++;
    jum_kar = i ;

    /* sandikan dan menampilkan ke layar */
    printf("\nHasil setelah disandikan: ");
    jum_penyandi = sizeof(konversi) / JUM_KOLOM;
    for(i = 0; i < jum_kar; i++)
    {
        karakter = kalimat [i];
        for(j = 0; j < jum_penyandi; j++)
        {
            if(karakter == konversi[j][0])
            {
                karakter = konversi[j][1];
                break; /* keluar dari for terdalam */
            }
            if(karakter == konversi[j][1])
            {
                karakter = konversi[j][0];
                break; /* keluar dari for terdalam */
            }
        }
        putchar (karakter);
    }
    printf("\n\n");
}

```

Contoh Eksekusi :

Masukkan sebuah kalimat dan akhiri dengan ENTER

Kemudian kalimat tsb akan saya sandikan

Kalimat : Saya belajar

Hasil setelah disandikan: Otyt bmltutr

Pada program tersebut ,

```
sizeof(konversi) / JUM_KOLOM)
```

merupakan ungkapan untuk memperoleh banyaknya baris (ukuran dari dimensi pertama) dalam array **konversi**. Dengan cara ini tabel konversi dapat diperluas tanpa perlu memberitahu banyaknya jumlah dimensi pertama secara eksplisit.

7.7 Array Sebagai Parameter

Array juga dapat dilewatkan sebagai parameter fungsi. Sebagai contoh ditujukan pada program **sorting.c**. Program digunakan untuk memasukkan sejumlah data, kemudian data tersebut diurutkan naik (*ascending*) dan dicetak ke layar.

Untuk melakukan sorting (proses pengurutan data), cara yang dipakai yaitu metode *buble sort* (suatu metode pengurutan yang paling sederhana, dan memiliki kecepatan pengurutan yang sangat lambat).

Algoritma pada metode pengurutan ini adalah sebagai berikut :

1. Atur *i* bernilai 0
2. Bandingkan $x[i]$ dengan $x[j]$, dengan *j* berjalan dari $i + 1$ sampai dengan $n-1$.
3. Pada setiap perbandingan, jika $x[i] > x[j]$, maka isi $x[i]$ dan $x[j]$ ditukarkan
4. Bila $i < (n - 1)$, ulangi mulai langkah 2.

Catatan: *i* = indeks array

x = nama array untuk menyimpan data

n = jumlah data

Algoritma berlaku untuk pengurutan menaik (*ascending*). Untuk pengurutan menurun (*descending*), penukaran dilakukan jika $x[i] < x[j]$.

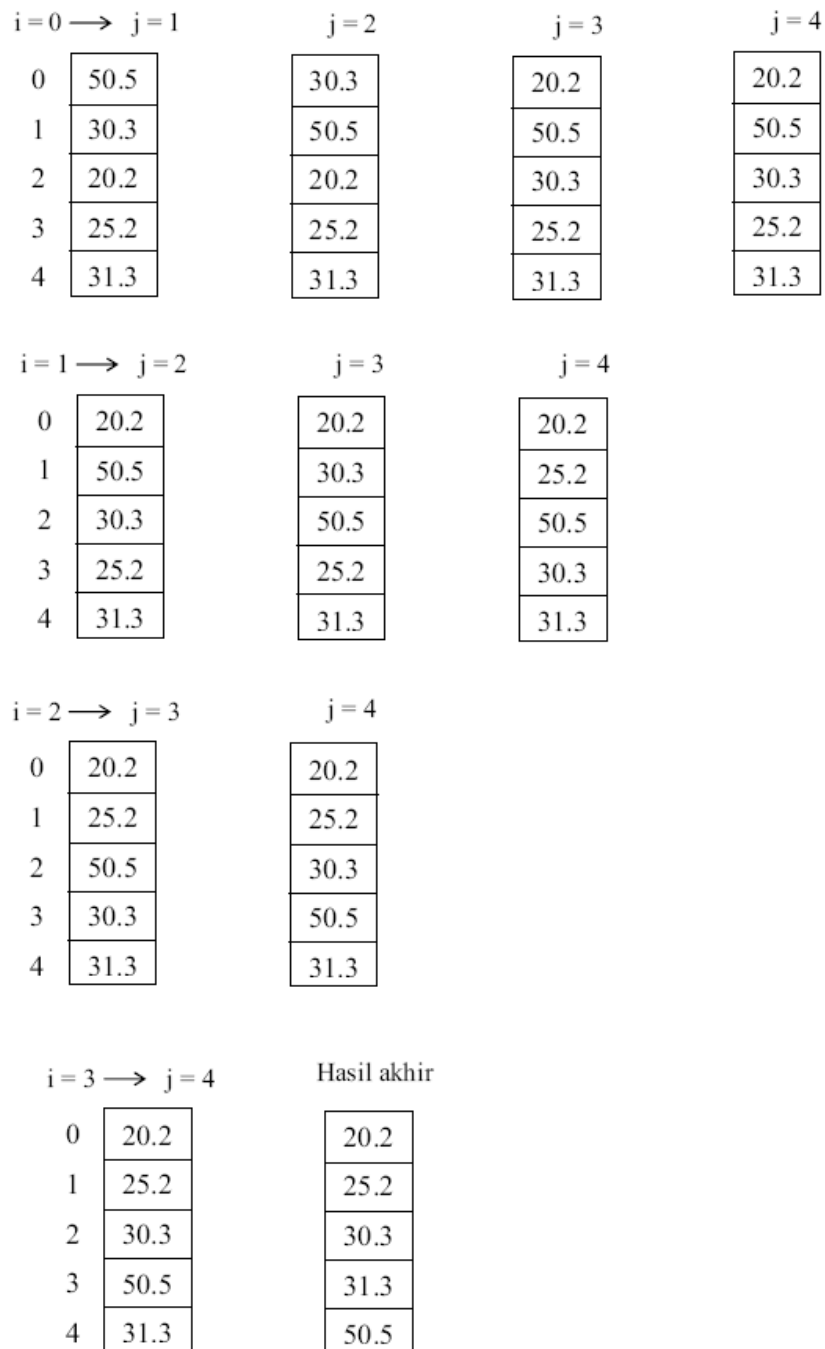
Penjelasan proses pengurutan terhadap 5 buah data ditunjukkan pada Gambar 7.6.

Data semula (sebelum pengurutan) adalah

50,5 30,3 20,2 25,2 31,3

Setelah pengurutan menaik (secara ascending), hasil yang diharapkan berupa

20,2 25,2 30,3 31,3 50,5



Gambar 7.6 Proses pengurutan data secara ascending dengan metode Bubble Sort

```

/* File program : sorting.c */
#include <stdio.h>

#define MAKS 20
void pemasukan_data(float [], int *);
void pengurutan_data(float [], int);
void penampilan_data(float [], int);
main()
{
    float data[MAKS];
    int jum_data;
    pemasukan_data(data, &jum_data);
    pengurutan_data(data, jum_data);
    penampilan_data(data, jum_data);
}

void pemasukan_data(float x[], int *pjumlah)
{
    int jum, i;
    printf("Jumlah data = ");
    scanf("%d", &jum);
    for(i=0; i<jum; i++)
    {
        printf("Data ke-%d : ", i+1);
        scanf("%f", &x[i]);
    }
    *pjumlah = jum;
}

void pengurutan_data(float x[], int jumlah)
{
    int i, j;
    float smtr;
    for(i=0; i<jumlah-1; i++)
        for(j=i+1; j<jumlah; j++)
            if(x[i] > x[j]) /* penukaran data */
            {
                smtr = x[i];
                x[i] = x[j];
                x[j] = smtr;
            }
}

```

```

}

void penampilan_data(float x[], int jumlah)
{
    int i;
    printf("\nData setelah diurutkan :\n\n");
    for (i=0; i<jumlah; i++)
        printf("Data ke-%d : %g\n", i+1, x[i]);
}

```

Contoh Eksekusi :

Jumlah data = 5

Data ke-1 : 50.5

Data ke-2 : 30.3

Data ke-3 : 20.2

Data ke-4 : 25.2

Data ke-5 : 31.3

Data setelah diurutkan

Data ke-1 : 20.2

Data ke-2 : 25.2

Data ke-3 : 30.3

Data ke-4 : 31.3

Data ke-5 : 50.5

Pada fungsi **main()**;

- `float data[MAKS];`

Merupakan instruksi untuk mendeklarasikan array data dengan elemen bertipe float dan jumlahnya sebanyak MAKS elemen.

- `pemasukan_data(data, &jum_data);`

Merupakan instruksi untuk memanggil fungsi **pemasukan_data ()**. Pada pemanggilan fungsi, data merupakan array. Yang perlu diperhatikan, parameter array ditulis tanpa diawali dengan &, sekalipun tujuan dari pemanggilan fungsi yaitu untuk mengisi data ke array. Sebab nama array tanpa kurung siku dalam parameter fungsi berarti "alamat dari elemen indeks ke-0 dari array tersebut".

Sedangkan **&jum_data** berarti "alamat dari jum_data". Tanda & harus disertakan sebab variabel **jum_data** akan diisi oleh fungsi **pemasukan_data()**.

- `pengurutan_data(data, jum_data);`

Merupakan instruksi untuk menjalankan fungsi **pengurutan_data ()**, dalam hal ini **data** dilewatkan ke fungsi dengan referensi (memberikan alamat array), karena memang hal ini merupakan cara satu-satunya untuk melewatkan array. Sedangkan jumlah data dilewatkan ke fungsi dalam bentuk nilai (pemanggilan dengan nilai).

- `penampilan_data(data, jum_data);`

Merupakan instruksi untuk memanggil fungsi **penampilan_data()**.

Pada fungsi untuk pemasukan data, pengurutan data maupun penampilan data,

```
data[i]
```

menyatakan elemen **data** ke-i.

Beberapa hal tambahan yang perlu diketahui:

- Untuk menyatakan alamat dari suatu elemen array, bentuk umumnya adalah

```
&nama_array[indeks]
```

Misalnya,

```
&data[1]
```

menyatakan alamat dari elemen ke-1. Adapun

```
&data[0]
```

sama saja dengan:

```
data
```

- Suatu array berdimensi satu dalam parameter formal dideklarasikan dengan bentuk

```
tipe nama_array[]
```

dengan di dalam tanda kurung siku tak disebutkan mengenai jumlah elemen. Jumlah elemen dinyatakan dalam parameter tersendiri (atau dinyatakan dalam bentuk variabel eksternal). Untuk array berdimensi lebih dari satu, kurung siku terkirilah yang kosong.

Kesimpulan :

- Array adalah kumpulan dari nilai-nilai data bertipe sama dalam urutan tertentu yang menggunakan sebuah nama yang sama.
- Nilai-nilai data di suatu array disebut dengan elemen-elemen array.
- Letak urutan dari elemen-elemen array ditunjukkan oleh suatu *subscript* atau indeks.
- Array bisa berupa array berdimensi satu, dua, tiga atau lebih.
- Data array akan disimpan dalam memori yang berurutan, dengan elemen pertama mempunyai indeks yang bernilai 0.
- Sebuah array dapat diinisialisasi sekaligus pada saat dideklarasikan, caranya saat mendeklarasikan array, nilai-nilai yang diinisialisasikan dituliskan di antara kurung kurawal ({}) yang dipisahkan dengan koma.
- Array juga dapat dilewatkan sebagai parameter fungsi.

Latihan :

Buatlah potongan program untuk soal-soal di bawah ini

1. Deklarasikan sebuah variabel array (misalkan nama variabelnya = **letters**) yang mengalokasikan untuk 10 elemen bertipe char.
2. Masukkan karakter 'Z' pada elemen yang ke-empat dari array **letters**.
3. Gunakan loop **for** untuk menghitung nilai akumulasi (total) dari seluruh isi sebuah array integer (*array of int*, misalkan namanya = **numbers**) yang memiliki 5 elemen.
4. Deklarasikan sebuah array multidimensi yang elemennya bertipe float (*array of float*, misalkan namanya = **balances**) yang memiliki 3 baris dan 5 kolom.
5. Gunakan loop *for* untuk menghitung nilai total dari seluruh isi array **balances** di atas.
6. Deklarasikan sebuah array karakter (*array of char*, misalkan namanya = **words**) dan sekaligus inisialisasikan dengan nilai string = "Hello"
7. Deklarasikan sebuah array karakter (*array of char*, misalkan namanya = **stuff**) dengan alokasi 50 elemen. Selanjutnya isikan dengan nilai string = "Welcome" pada body programnya (bukan pada saat deklarasi)

8. Gunakan statemen *printf()* untuk menampilkan (di layar) isi elemen ke-tiga dari sebuah array integer (*array of int*, misalkan namanya = **totals**)
9. Gunakan statemen *printf()* untuk menampilkan (di layar) string yang merupakan isi dari sebuah array karakter (*array of char*, misalkan namanya = **words**)
10. Gunakan statemen *scanf()* untuk membaca string masukan dari keyboard dan memasukkannya ke array **words**.
11. Tuliskan loop *for* untuk membaca 5 karakter (gunakan statemen *scanf()*) dan memasukkannya ke dalam array **words**, mulai dari indeks 0.